

LLM 基础：面向 Harness Engineering 的实践者教材

这是一份讲解大语言模型机制与行为边界的实践者教材。

18 sections · generated from Markdown sources

前言

为什么需要这本书

通过 API 调用大语言模型并不难，但聊天界面会隐藏那些塑造模型行为的机制。Tokenization 会改变模型实际接收到的内容；训练决定哪些模式被写入参数；有限的 context 会限制哪些信息能够影响回答；decoding 则把 next-token probability 转化为生成文本。如果缺少对这些机制的理解，人们很容易把流畅的输出误认为最新知识、可靠推理或已经核实的事实。

本书解释这些机制以及它们形成的边界。它的目的不是让模型的每一次回答都变得可预测，而是让模型行为不再那么神秘，并为读者提供一套精确的分析语言。

目标与读者

本书面向使用大语言模型的软件工程师和技术读者，并沿着一条面向工程实践的 LLM 基础路线展开：token、next-token prediction、Transformer attention、训练数据与 scaling、inference 与 decoding、post-training、prompting、context、知识边界、检索、工具调用和评估。

读完本书后，读者应该能够从 token、训练、context、decoding 和外部证据这些角度解释模型行为，也应该能够区分哪些性质属于模型本身，哪些性质属于围绕模型构建的应用。

素材来源

本书主要整理自 Andrej Karpathy 的 [Intro to Large Language Models](#) 和 [Deep Dive into LLMs like ChatGPT](#)。第一期讲座用两个组成部分来理解 LLM：参数，以及运行这些参数的代码。第二期讲座进一步展开数据、tokenization、训练、inference、post-training 和实用的心智模型。本书把这些讲座整理成一条书面学习路线，并在基础论文能够为具体概念提供依据时补充相应引用。

先修要求与非目标

读者应该熟悉 API、测试和日志等基本软件工程概念，不需要预先掌握深度学习数学。在数学有助于理解时，本书只引入理解相关机制及其后果所需的内容。

本书不教读者如何训练 frontier model，不比较当前所有模型供应商，也不完整综述每一种 alignment 方法。这些目标要么变化很快，要么需要更深入的篇幅。本书关注的是一套稳定的基础，用来理解工程师和技术读者在实践中遇到的模型。

稳定的模型性质

接下来的章节围绕几项模型侧的事实展开：

- 语言模型接收和生成的是 token，而不是直接接收和生成词语或意义。
- 模型的 context 有限，而且不是持久记忆。
- 训练期间学到的知识分布在参数中；这些参数不是实时、权威的数据库。
- Decoding 规则从 next-token probability 中选择 token，因此生成方式的配置也是最终行为的一部分。

后续章节会进一步细化这些表述，说明这些简化说法的边界，并解释外部证据如何进入模型的 context，而不会因此变成存储在模型参数中的知识。

配套教材

配套教材 [Agent Harness：实践者教材](#) 介绍围绕模型构建的系统，包括 context 管理、工具、沙箱、工作流模式和评估。本书则建立这些系统设计必须考虑的模型机制与行为边界。两本书都可以独立阅读；当主题跨越这条边界时，读者可以沿配套链接继续了解。

本书的章节地图和推荐阅读顺序见 [README](#)。

第 1 章：把 LLM 看成 Token 机器

最有用的第一近似很简单：大语言模型接收一串 token，并为下一个可能出现的 token 计算概率。它并不直接看见单词、文件、网页或用户，而是接收表示当前上下文的 token ID 序列。

Token 输入，概率输出

对于给定的 token 上下文，模型会为词表中的每个 token 计算一个分数。运行时将这些分数转换成下一个 token 的概率分布。从这个意义上说，LLM 是一个从 token 序列映射到下一个 token 概率的参数化函数。

Karpathy 在短讲座中使用了一个刻意去神秘化的框架：一个训练好的模型可以粗略理解为两个文件，一个保存参数，另一个保存知道如何运行这些参数的代码 ([Intro to LLMs, around 00:00:24](#))。真实实现包含更多机制，但这一近似揭示了基本接口：输入 token 上下文，输出下一个 token 的概率分布。

序列如何生成

一次计算通常不会直接产生完整回答。生成过程会重复以下基本操作：

1. 计算当前 token 上下文。
2. 产生下一个 token 的概率分布。
3. 按照某种解码规则选择一个 token。
4. 将这个 token 追加到上下文。
5. 重复以上步骤，直到满足停止条件。

这里使用“选择”一词很重要。贪心解码选择概率最高的 token，随机解码则从概率分布中采样。两者都是解码规则，但贪心选择不属于采样。[Inference](#) 和 [sampling](#) 会详细说明这个循环及其停止条件。

架构、参数、运行时与应用层

四个层次经常被混在一起：

- **架构**定义计算的形状，例如 decoder-only Transformer。
- **参数**是通过学习得到、使架构具备具体特性的数值。
- **运行时**加载参数、计算模型，并应用解码规则。
- **应用层**准备输入、调用运行时，并解释或呈现输出。

两个模型可能共享同一种架构，但包含不同的参数。反过来，两个应用也可能调用同一个 checkpoint，却因为准备了不同的上下文，或以不同方式解释返回的 token，而呈现不同的用户体验。

Karpathy 把参数描述为可以复制、下载并由运行时代码加载的普通文件，使这个概念变得具体 ([Intro to LLMs, around 00:01:35](#))。参数文件的存储大小大致取决于参数数量和数值精度。一个 700 亿参数的模型，如果每个参数用 2 bytes 存储，仅参数本身就需要约 140 GB，尚未计入运行时开销。

参数从哪里来

参数由训练产生，而不是由 inference 循环产生。预训练期间，优化过程反复修改参数，以降低模型在大规模 token 序列集合上的下一个 token 预测误差。创建大型 checkpoint 可能需要大量数据、算力和时间，而运行已有 checkpoint 则是更常规的操作 ([Intro to LLMs, around 00:03:59](#)、[00:05:16](#))。

Post-training 从预训练 checkpoint 开始，进一步修改已经学得的参数，或加入学得的 adapter 参数，从而塑造遵循指令、答案格式和拒答等倾向。其结果是更新后的 checkpoint 或另一组新增的学得权重，而不是让原始参数文件保持不变。

大多数模型使用者不会自己创建 base checkpoint，而是通过 API、inference provider 或本地运行时调用已有 checkpoint。无论 inference 在哪里运行，这一区分都成立：训练产生学得的参数，运行时使用这些参数计算下一个 token 的概率。

参数不是数据库

参数编码的是分布式统计结构和学得的行为。它们不是可以按键查询、作为精确记录检查或逐条更新事实的数据库行。模型可以复现训练中学到的事实和模式，但参数文件并不是权威数据库，也不会为每个生成的断言附带内置来源。

这一区分说的是表示方式，而不是模型能否回答事实问题。模型可以从学得的参数中流畅地回忆信息，但这不会让参数变成记录存储系统。

Token 不会产生外部副作用

模型的直接输出是 token。生成的后续文本可能描述读取文件、发送消息或采取其他动作，但生成这段描述并不会让动作真正发生。外部软件必须解释输出并执行相应操作。

[推理、工具与 agent](#) 会再次讨论这条边界：模型可以生成一个拟议调用，外部系统执行它，并可能返回 observation。动作提议和外部效果始终是两个不同的事件。

一次模型调用不等于完整应用

Karpathy 的 deep dive 一开始就问：文本框背后是什么，生成出来的词又从哪里来 ([Deep Dive, around 00:00:28](#))？在模型边界上，答案是 token 生成循环；在应用边界上，还包括构造输入、调用运行时和处理输出的软件。

讨论 *模型行为* 仍然是有意义的：这个说法可以指特定条件下学得的倾向、概率分布或生成序列。但可观察到的应用行为还可能包含界面呈现、存储状态和外部效果。这些额外结果并不是一次模型调用单独产生的。

接下来的章节会进一步展开模型侧图景：[Tokenization](#) 解释文本如何变成 token ID，[下一个 token 预测](#) 解释训练目标，[Transformer attention](#) 解释核心架构，[inference](#) 和 [sampling](#) 解释解码。模型边界之外的系统设计属于配套教材 [Agent Harness](#)，尤其是其中关于 [harness 边界](#)、[状态](#)、[权限与副作用](#) 和 [评估](#) 的章节。

要点

- LLM 将 token 上下文映射为下一个 token 的概率。
- 序列生成会重复下一个 token 预测和 token 选择，直到满足停止条件。
- 架构、参数、运行时与应用层是彼此不同的层次。
- 预训练和 post-training 产生学得的参数；inference 使用这些参数。
- 参数是分布式的学得表示，而不是可查询数据库。
- 模型生成的 token 本身不会引发外部动作。

第 2 章：Tokenization

语言模型处理文本之前，tokenizer 会先把文本转换成一串整数 token ID。一个 token 可能对应完整单词、单词的一部分、标点、空白与相邻字符的组合、一个 byte，或其他学到的单元。Token 边界是特定 tokenizer 的属性，并不是通用的语言学边界。

Karpathy 花时间演示 tokenizer，是因为这些例子能解释一些看似意外的行为：前导空格可能改变 token ID，大小写可能改变切分，罕见字符串可能展开成许多片段 ([Deep Dive, around 00:12:07](#))。Tokenization 的直接结果是序列长度。序列长度进而影响一个 context window 能容纳多少文本；具体到某个模型和服务时，它还可能影响计算量、延迟或计费用量。

从文本到 Token ID

从高层看，输入路径是：

文本 → token 片段 → token ID → embedding 向量

Tokenizer 完成前两个转换。它按照固定词表和编码规则切分文本，再查出每个片段对应的整数 ID。模型用这些 ID 从学得的 embedding table 中选择对应行。模型实际操作的是得到的向量，而不是原始字符。

解码使用反向映射：把 token ID 映射回片段，再将它们拼接成文本。但这并不意味着每一种 tokenizer 都能让所有输入原样往返。可选的 normalization 可能在切分前改变文本，解码时也可能省略控制 token。如果 byte-level tokenizer 的 byte 映射和解码规则始终一致，通常可以精确保留普通输入文本。

Tokenizer 是模型规格的一部分。两个模型可以把同一个字符串切成不同片段、分配不同 ID，并得到不同的序列长度。因此 token ID 只有和匹配的 tokenizer 及词表一起使用时才有意义。

为什么需要 Subword Token

只包含完整单词的词表很脆弱。人名、词形变化、代码标识符、URL、科学记数法、emoji 和多语言文本都会不断产生未见过的词。只使用字符的词表可以表示这些输入，但通常会形成长得多的序列，也让模型失去把高频多字符模式作为单一单元复用的机会。

Subword tokenization 是一种折中。高频字符串可以拥有自己的 token，低频字符串则由更小的片段组合而成。Sennrich、Haddow 和 Birch 说明了如何在开放词表神经机器翻译中用 subword unit 分解罕见词和未见词 ([Neural Machine Translation of Rare Words with Subword Units](#))。

Byte Pair Encoding (BPE)、WordPiece 和 Unigram 都属于相关的 subword 方法，但它们学习和切分文本的方式并不完全相同。BPE 训练通常从一套基础单元开始，反复为选中的相邻单元对加入新 token。基础单元取决于具体 tokenizer：经典形式可能从字符或符号开始，而 **byte-level BPE** 从 byte 的某种表示开始。

这个区别很重要。如果一个 byte-level BPE tokenizer 保留完整的 byte alphabet，它就能通过 UTF-8 byte 编码任意 Unicode 文本；当没有更大的已学片段匹配时，就回退到更小的 byte 单元。不能因此声称所有 BPE tokenizer 都从原始 byte 开始，也不能声称所有 subword tokenizer 都必然不存在 unknown token。其他 tokenizer 可能使用 unknown token、字符覆盖规则，或单独的 byte-fallback 机制。

Tokenizer 如何训练和使用

Tokenizer training 发生在 tokenizer 被用来编码模型输入之前。训练者先收集有代表性的文本语料，并决定 normalization、空白处理、pre-tokenization、基础 alphabet、special token 和词表大小。训练算法随后学习词表，以及特定方法所需的 merge rule 或 token score。模型训练一旦开始，这个 tokenizer 通常就会固定下来，使每个 token ID 始终对应同一个 embedding 行。

训练 BPE 时，算法统计相邻单元，并反复加入选定的 merge，直到达到目标词表大小或其他停止条件。Unigram tokenization 则从大量候选片段开始，学习它们的概率，在保留高概率切分的同时逐步裁剪词表。这两种过程都能产生 subword 词表，但得到的边界不必相同。

编码时，tokenizer 通常会执行以下步骤的某种组合：

1. 如果 tokenizer 定义了 normalization 规则，就先对文本进行 normalization。
2. 按照空白和 pre-tokenization 规则切分或标记输入。
3. 使用学到的词表和算法进行 segmentation。
4. 把每个片段替换成它在词表中的 ID。
5. 加入模型输入格式要求的 special token。

不同实现可能合并或省略其中一些步骤，因此应以 tokenizer 本身为准。解释生成出的 token ID 时，也必须使用它对应的 decoder。

如何阅读 Tokenization 示例

必须用一个具体 tokenizer 才能确定 token 边界。下表展示的是可能的切分，不代表每个模型都会如此切分：

文本	一种可能的切分	说明
world 与 world	两种形式使用不同片段	前导空格可能和相邻文本一起编码。
unhappiness	un + happi + ness	熟悉的 subword 片段可以组合成低频词。
HTTPResponse2	较大片段与较小片段的组合	大小写、标识符和数字会改变边界。
你好	一个片段、多个字符片段，或 byte fallback	语言覆盖程度和 tokenizer 的基础单元都会产生影响。

Karpathy 用大小写、前导空格和细小标点变化演示了同一个原理 ([Deep Dive, around 00:12:33](#))。换行和缩进也可能改变代码的 token 序列。调试工具有时会用人为标记表示空格或 byte；这些标记描述的是 tokenizer 的内部表示，不一定是输入中的字符。

因此，小的格式变化可能产生不同的 token 序列。它们不保证一定带来明显的行为变化，但模型接收到的确实已经是另一串 ID。

Token 数不是单词数

模型的序列限制以 token 计量，而不是以单词、文件、消息或字符计量。在熟悉的英文 prose 中，每 token 对应多少字符的粗略估算可以用于快速预判，但不能当成可靠的换算规则。

代码、表格、数字、URL、重复空白、罕见符号和不同书写系统的切分方式都可能不同。结果还取决于 tokenizer 训练时对某种语言或文本风格的覆盖程度。因此，字符数或词数相同的两段文本不一定会占用相同数量的 token。

要得到精确数量，应使用目标模型的 tokenizer 编码实际文本，再统计返回的 ID。Context window 通常必须同时容纳输入和生成的 continuation；[第 9 章](#) 会详细解释这个共享的序列限制。计算量和计价策略是模型及供应商针对这些计数定义的结果，并不是 tokenization 本身的属性。

文本 token 数也不能用来估算图片、音频或其他多模态输入。这些输入使用模型特定的 encoder 和计量规则。

Special Token 和 Chat Template

Special token 是为结构或控制用途保留的词表条目。它们可能标记序列开始或结束、分隔文档，或划分消息与角色，具体取决于模型。它们仍然是 token ID，但其含义来自模型训练时使用的格式，而不是普通书面语言。

Chat API 可能接收消息列表，但模型最终得到的仍是一段编码后的序列。**Chat template** 会把消息角色和内容渲染成模型预期的格式，通常还会插入 special token 和固定分隔符。Assistant model 在 [post-training](#) 中学习这些约定。

Karpathy 在 deep dive 后面再次讲到 tokenization，说明对话和其他模型输入一样，也必须表示成 token ([Deep Dive, around 01:05:03](#))。渲染后的 template 会产生消息正文中看不见的 token 开销。因此，即使用了正确的文本 tokenizer，只统计 content 字段也可能低估请求长度。精确计量必须同时使用目标模型的 chat template 和 tokenizer；不同模型家族或版本的格式与开销可能不同。

字符级能力缺口

对多数文本而言，tokenization 并不会为每个字符提供一个模型位置。一个词可能占一个 token，也可能占多个 subword token，还可能回退成字符或 byte 序列。因此，需要稳定访问单个字符的操作——例如数字母、反转陌生字符串或追踪精确拼写——可能会比表面上更困难。

这是一种倾向，不是绝对的能力缺失。模型可以学习拼写模式、推断 token 内部包含的字符，有时也能正确完成这些任务。不过，tokenization 仍然是表示方式与逐字符操作难以对齐的一个原因。Karpathy 用拼写和字母计数示例说明了这种错位 ([Deep Dive, around 02:01:11](#))。

数字也可能被切成长度不一的片段，但 token 边界不能完整解释算术错误。训练数据、next-token objective、学得算法的局限，以及保存中间状态的难度都可能造成影响。

常见 Token 计量陷阱

模型特定的编码方式会带来几种反复出现的错误：

- 根据单词数或字符数估算，并把估算值当成精确上限。
- 使用另一个模型家族或 tokenizer 版本来计数。
- 只统计消息正文，没有应用 chat template 及其 special token。
- 只计量 prompt，却忽略生成 token 也会占用可用序列。
- 假设 prose、源代码、URL 和多语言文本具有相同 token 密度。
- 把文本 token 估算方式用于图片、音频或其他多模态输入。

截断还会带来一个相关问题。按字符截取不一定符合目标 token 预算，按 token 截取则可能在解码后切断句子或其他有意义的单元。Token 数决定序列能否放得下，却不能决定哪些内容在语义上可以安全删除。

要点

- Tokenizer 把文本映射成模型特定的 token ID；embedding lookup 再把 ID 转成模型处理的向量。
- Subword 方法在复用高频片段和组合陌生字符串之间取得平衡。
- BPE 是一类方法，只有 byte-level 变体才必然从 byte 表示开始。
- 序列长度取决于具体 tokenizer、文本、special token 和 chat template，而不是单词数。
- Tokenization 会让字符级操作变得不自然，但并非所有拼写或算术错误的唯一原因。

第 3 章：Next-Token Prediction

自回归语言模型的预训练目标非常简单：给定前面的 token，预测下一个 token。在每个位置，模型会为词表里的每个 token 分配概率；训练会奖励模型给真实后续 token 分配更高的概率。Karpathy 用从大数据集中抽取 token window、反复预测接下来会出现什么的方式演示了这个过程 ([Deep Dive, around 00:15:36](#))。

这个目标为预训练提供学习信号。它本身不会赋予模型权威的事实存储、意图或 assistant 人设。

Next-Token 目标

假设一个训练序列以这句话开头：

The capital of France is Paris.

它提供了一系列预测目标：

The	-> capital
The capital	-> of
The capital of	-> France
The capital of France	-> is
The capital of France is	-> Paris

实际单位是 token 而不是单词，因此真实边界可能和这个简化展示不同。同一个目标适用于 prose、代码、对话、数学推导、表格和 Markdown。无论文本采用什么形式，训练目标都表示成 continuation token，而不是单独编写的符号规则。

训练文本变成 Token 序列

训练开始前，源文本会经过过滤、转换，再由 tokenizer 编码。得到的 token ID 会被组织成模型能够处理的长度有限的序列。每个序列都会为训练提供许多 prefix-target 关系。

在 Karpathy 的小型演示中，清洗后的数据集被表示成一条很长的 token 数组，训练窗口从中采样 ([Deep Dive, around 00:14:34](#))。这是一种有用的教学方式，并不是对生产训练的普遍描述。生产 pipeline 可能保留文档边界、插入 boundary token、拆分长文档，或把多个文档片段打包成固定长度的序列。具体构造方式决定模型会遇到哪些跨片段 transition。

不变的核心是：训练在带有 next-token target 的 token 序列上进行。它不要求把整个语料库变成一条语义上毫无边界的长流。

Loss 和 Gradient 更新

对一个 target token 而言，loss 是模型为它分配的概率取负对数。正确 token 的概率高，loss 就低；概率低，loss 就高。对所有预测位置的这个数值取平均，就得到 cross-entropy loss；在这里它也称为 negative log-likelihood。

使用自然对数时，perplexity 是平均 cross-entropy 的指数。它把 loss 放到“有效选择数”的尺度上：perplexity 为 10，对应的平均不确定程度等同于从 10 个选项中均匀选择。Karpathy 把 loss 描述为训练过程试图降低的单一数字 ([Deep Dive, around 00:35:58](#))。

训练循环是：

1. 构造一批 token 序列及其 next-token target。
2. 运行模型，在每个预测位置产生 probability distribution。
3. 计算 target token 的平均 loss。
4. 在模型中反向传播 gradient。
5. 使用 optimizer 更新参数。
6. 在许多 batch 上重复。

没有人会为每种语法模式、事实或代码惯例分别编写规则。Gradient update 会改变许多参数，让类似训练分布的 continuation 变得更可能。

记忆与泛化

这个目标不会强迫模型在记忆和泛化之间二选一。模型可以记住一些完全相同或近似相同的序列，尤其是重复出现或特征鲜明的序列。它也可以学习许多样本共享的规律，再把这些规律应用到训练时没有见过的序列上。后一种行为就是泛

化。

Karpathy 使用 compression 作为直觉：一个能很好预测文本的模型，捕捉了数据中的结构 ([Deep Dive, around 00:50:22](#))。这个类比是有损的，而不是像 zip 一样的无损压缩。参数可以编码广泛模式、关联和一些精确字符串，却不会把每个来源都保存成可恢复的记录。

这也解释了为什么参数化知识并不是权威或自动保持最新的信息源。它反映训练分布及其时间覆盖范围，可能混合相互冲突的模式，而且不会天然保留生成结论的来源。一个看似合理的 continuation 并不等于经过验证的事实。

为什么预测会产生广泛能力

为了改进在网页、书籍、代码、论文和对话上的 next-token prediction，模型需要形成对许多潜在规律的表示。语法有助于预测句子，事实关联有助于预测百科式段落，程序结构有助于预测代码，对话惯例有助于预测 conversation，算术和推理模式则有助于预测带步骤的示例。

GPT-3 论文表明，足够大的自回归模型可以根据 context 中的指令或示例适配许多任务，而不需要针对任务更新参数 ([Language Models are Few-Shot Learners](#))。这就是 in-context learning，第 8 章 会进一步讨论。

这些结果并不能证明模型在所有意义上都具有人类式理解，也不意味着每种能力都会同等或可靠地出现。它们说明，同一个预测目标可以产生支持许多下游行为的内部表示。

有能力不等于有意图

预训练通过改变参数来降低 prediction loss。它不会让模型内在地承诺做到 helpful、truthful 或 safe。Base model 学到的是有帮助的回答、有害段落、正确解释、错误以及许多其他文本的统计模式。模型会继续哪种模式，取决于它学到的分布和收到的 prefix，而不是独立形成的意图。

因此，改变 prefix 可以改变可能 continuation 的分布。这是 prompting 和 in-context learning 的基础，但通过 prefix 进行 conditioning 并不会安装新的训练目标或持久意图。

Base Model 与 Assistant Model

Next-token pretraining 的直接结果是 base model。它可以补全文本、模仿格式、回答一些问题并复现许多学到的模式，但不一定经过优化，能始终以 helpful conversational assistant 的方式回应。

Post-training 会进一步改变模型的输出分布，提高 instruction following、conversational helpfulness 和 learned refusal 等行为的概率。这些 assistant 行为是学到的 response tendency，不是存在内在意图的证据，也不能保证真实性或安全性。

要点

- 自回归语言模型通过预训练，根据前面的 token 预测每一个 next token。
- 训练使用长度有限的 token 序列；把整个数据集表示成一条长流是说明性实现，不是普遍要求。
- Cross-entropy loss 和 gradient update 会同时产生记忆与泛化。
- 广泛的预测能力不会创造内在意图或权威知识。
- Base model 来自预训练；assistant 行为则由 post-training 进一步塑造。

第 4 章：Transformer 与 Attention

现代大多数 LLM 都基于 *Attention Is All You Need* 提出的 Transformer 架构 (Vaswani et al., 2017)。Karpathy 在入门讲解中也指出，这类模型背后的神经网络架构是 Transformer (Intro to LLMs, around 00:11:40)。这一架构解释了 token 序列如何变成依赖上下文的预测。

输入与输出

语言模型从 token ID 开始。Embedding table 把每个 ID 映射为一个向量，于是输入序列就变成了向量序列。模型还必须表示顺序：没有位置信息时，同一组 token 以不同顺序出现会很难区分。

不同 Transformer 家族用不同方法编码位置，例如 learned absolute position embedding、sinusoidal encoding、relative position bias，以及在 attention 内应用的 rotary position embedding。因此，位置是模型序列表示的一部分，但不一定总是以直接加到 token embedding 上的形式存在。

这些向量会经过一组堆叠的 Transformer block。每个 block 都为各个位置产生新的表示。最后一个 block 之后，学得 projection 把每个位置的表示映射为词表 logits。进行 next-token training 时，这些 logits 会与每个可预测位置的后续 token 比较；生成时，最后一个位置的 logits 定义用于选择下一个 token 的分布。

Karpathy 将这一流程描述为：token 经过重复的 Transformer block，直到网络产生 next-token prediction (Deep Dive, around 00:23:24)。模型并不是把序列解析成一组显式变量，而是在反复转换向量表示。

原始 Transformer 与 Decoder-Only LLM

原始 Transformer 是面向机器翻译等 sequence-to-sequence 任务的 encoder-decoder 架构。Encoder 为输入序列构建表示；decoder 既对输出前缀使用 causal self-attention，也通过 cross-attention 使用 encoder 的表示。

许多 autoregressive LLM 改用 decoder-only 架构。它们没有单独的 encoder 或 cross-attention 阶段，而是让同一组堆叠层在 causal constraint 下处理前缀，并预测紧随其后的 token。训练时，每个位置的正确前缀已经存在，所以可以并行计算许多 next-token prediction；生成时，则逐个追加 token。

因此，“Transformer”指的是一族架构，而不是一种固定布局。不同模型家族还会采用不同的 normalization placement、position method、attention variant、activation function 等细节。它们共享的核心思想是：用一组学得的 block，把 token 序列转换成上下文化表示。

Self-Attention

Self-attention 让一个位置能够混合同一序列中其他位置的信息，从而使 token 表示依赖其上下文。例如，问题中的词可以与前面的 source text 发生交互，一行代码也可以与更早的变量定义发生交互。

Attention computation 是学得的、柔性的操作。它不会按固定规则检索一条符号记录或复制某个事实，而是生成 value vector 的加权混合；这些结果还会与其他信号结合，再由网络的其余部分继续转换。

Query、Key 与 Value

在单个 attention head 内，学得 projection 会把每个输入表示转换成三种向量：

- **Query** 表示当前位置在寻找什么。
- **Key** 表示一个可用位置可以怎样被匹配。
- **Value** 表示这个可用位置可以贡献什么信息。

对于输入矩阵 (X)，一个 head 会计算 projection ($Q=XW_Q$)、($K=XW_K$) 和 ($V=XW_V$)。其输出形式为

$$\text{Attention}(Q, K, V) = \frac{\text{softmax}((Q K^T) / \sqrt{d_k})}{M} V$$

其中 (M) 是 attention mask。点积产生匹配分数，softmax 把允许使用的分数转换成权重，加权求和则混合各个 value vector。

假设一段代码中包含 `user_count = ...`，随后执行到 `return`。某些 head 可能会给前面的定义相对较高的权重，并把与 `user_count` 有关的信息贡献给后面位置的表示。这只是对一次中间计算的直觉说明：通常并不是某一个 head 直接决定下一个 token 就是 `user_count`。在词表 logits 产生之前，多个 head、residual path、MLP 和许多层的输出都会共同参与。

对散文也需要保持同样的谨慎。用于回答问题的位置可能会给前面陈述相关事实的句子较高权重，但最终答案并不是由某一条 attention link 机械决定的。Attention 可能混合有用、无关或误导信息；仅凭较高的 attention weight，也不能证

明某个位置具有因果重要性。

Causal Mask

在 decoder-only language model 中，mask (M) 会阻止当前位置 attend 到更靠后的位置。不允许使用的分数会在 softmax 前被实际设为负无穷，因此对应权重变为零。位置 (i) 的表示可以使用位置不晚于 (i) 的 token，却不能使用它之后的 token。

Causal mask 让架构与 next-token prediction 对齐。训练时，模型可以处理完整序列，同时仍然阻止每个位置看到自己应该预测的 token；生成时，同一约束意味着新生成的 token 只能依赖已有前缀。

Multi-Head Attention

Multi-head attention 使用不同的 learned projection 并行运行多次 attention computation。各个 head 的输出会被拼接，再 projection 回模型的 residual stream。多个 head 使一层能够以多种方式并行组合不同位置的信息。

有些 head 会呈现可识别的模式，例如关注局部位置或追踪重复 token，但并不能保证所有 head 都具有清晰、可用人类语言描述的分工。它们的贡献可能互相重叠、随上下文变化，并与后续层发生交互。更合适的理解是：head 是分布式计算的一个组成部分，而不是独立的决策者。

Karpathy 在解释 Transformer 内各个位置如何通信时，重点指出了 attention block ([Deep Dive, around 00:24:29](#))。它为前文影响后续表示提供了路径，但不保证模型一定能正确找出并使用最相关的信息。

Transformer Block

Attention 只是 Transformer block 的一部分。典型的 decoder-only block 包含：

1. 一次 normalization。
2. Causal multi-head self-attention。
3. 一条 residual connection，把 attention 输出加回持续传递的表示。
4. 另一次 normalization。
5. 一个对各位置独立应用的 feed-forward network，通常称为 MLP。
6. 另一条 residual connection。

具体顺序因模型家族而异；例如，normalization 可能位于 sublayer 之前或之后。比确切变体更重要的是功能差异：attention 在 token 位置之间混合信息，MLP 则在每个位置内应用学得非线性变换。Residual connection 让信号经过许多 block 时得以保留和累积，normalization 则有助于保持计算稳定。

重复这些 block 会产生以可用前缀为条件的表示。各层不一定对应“先语法、后语义”这样的整齐阶段。特征可以分散在多个层与组件中，同一组件也可能针对不同输入参与不同计算。

分布式表示与可解释性

模型学到的知识和行为分布在许多参数与 activation 中。Karpathy 强调，数十亿参数以我们尚未从机制上完全理解的方式共同工作 ([Intro to LLMs, around 00:11:57](#))。因此，“某个事实存在哪里”通常没有简单答案。

特定预测也同样如此。Attention weight 揭示了内部计算的一部分，却不是完整解释：value vector、MLP、residual path 和后续层都会发挥作用。Mechanistic interpretability 研究这些计算，但现有方法通常还不能为每个生成 token 提供简单而完备的因果说明。

分布式表示也意味着模型知识不是一张可直接编辑的事实表。改动一个参数可能影响许多输入，而一种行为也可能依赖大量参数。网络仍然执行确定的数值计算，但其内部概念通常不会与人类标签一一对应。

架构与训练是两件事

架构定义网络可以执行的运算，训练则确定这些运算使用的参数值。随机初始化的 Transformer 拥有架构，却没有实用的语言行为。Pretraining 让参数拟合训练序列上的 next-token prediction，post-training 则可以进一步改变其回答分布。

这种区分可以避免把架构当成包罗一切的解释。Self-attention 为前文影响后续预测提供了路径，但训练决定模型学会识别哪些模式，以及使用这些模式的可靠程度。架构相近的两个模型，可能因为数据、目标、参数量和 post-training 不同而表现迥异。

Attention 成本

对 (n) 个 token 进行标准 full-sequence attention 时，每个位置都要与其他允许使用的位置形成分数，因此每个 head 的 score matrix 包含 $(O(n^2))$ 个元素。在模型宽度和层数不变时，训练以及处理 prompt（也称为 **prefill**）期间，attention 中随序列长度变化的计算量是平方级的。Causal mask 去掉了对未来位置的交互，但不会改变这一渐近复杂度。

这个结论专门针对 block 中的 attention 部分。Linear projection 和 MLP 同样消耗大量计算，在许多实际序列长度下甚至可能主导总运行时间。传统 attention 实现还可能需 $(O(n^2))$ 的临时内存来存放 attention score；memory-efficient kernel 可以避免显式生成完整矩阵，但并不会消除底层成对 attention 运算。

使用 **KV cache** 时，autoregressive decoding 呈现不同的计算特征。各层会保存已有 token 的 key 与 value。对于一个新 token，模型只计算新的 query、key 和 value，再让这个 query attend 到缓存的前缀。在模型维度固定时，这一步的 attention 计算量随前缀长度按 $(O(n))$ 增长，而不必为整个前缀重新计算 $(O(n^2))$ attention。KV-cache memory 则随缓存 token 数量线性增长。

生成多个 token 时，这些逐步线性的成本会累积：在长度为 (p) 的 prompt 之后生成 (m) 个 token，attention score work 大致与 $(mp + m^2)$ 成正比，此外还要加上 prefill 与每个 block 中的其他计算。架构与 kernel 优化可以改变常数，或改用近似、稀疏模式，但更长的序列仍会带来真实的计算、内存和延迟成本。

要点

- Decoder-only Transformer 让 token 与位置信息经过重复 block，最终得到 next-token logits。
- Causal self-attention 通过学得的 query、key 和 value projection，混合可用前缀的信息。
- 多个 head 和多层会共同贡献分布式的中间信号；单个 attention head 并不直接选择输出 token。
- Attention 能让相关前文产生影响，但不保证一定能正确选择或使用这些信息。
- 标准 prefill attention 的计算量随序列长度平方增长，而使用 KV cache 的 decoding 对每个新 token 使用线性 attention 计算和线性 cache memory。

第 5 章：训练数据与 Scaling

预训练质量取决于训练数据的分布、模型参数的数量与架构，以及用于优化这些参数的算力。这些因素会相互作用：只有训练过程提供足够且合适的数据与算力，更多参数才有用；更多 token 也只有在增加可学习信号时才有用。Karpathy 的 deep dive 从数据收集与过滤讲起，因为训练分布是决定模型能力和盲点的主要因素之一 ([Deep Dive, around 00:01:28](#))。

训练数据是构造出来的分布

大型预训练语料可能包含网页、书籍、代码、论文、讨论和文档，也可能包含重复模板、spam、抽取错误、过期论断、有毒材料、个人数据，以及对某些语言和领域的强烈偏置。因此，语料并不只是“整个 Web”，而是数据 pipeline 将收集到的来源转化为 token 序列分布之后的产物。

FineWeb 是这一过程的公开例子。Deep dive 用它说明如何把源自 Common Crawl 的网页转换成 web-scale 文本数据集；视频讨论的版本约有 15 万亿个 token ([Deep Dive, around 00:01:28](#))。原始网页除了有用文本，还包含导航菜单、cookie banner、boilerplate、重复镜像和抽取错误。

Dataset construction 通常包括：

- 收集网页快照和其他来源语料；
- 抽取并规范化文本；
- 识别语言和内容类型；
- 过滤低质量、不需要或敏感的材料；
- 删除完全重复和近似重复的内容；
- 为不同来源设置采样权重；
- 将选出的文本 token 化并打包用于训练。

由此得到的分布是有意构造出来的。它既不是人类知识的均匀样本，也不是真理的直接表示。

过滤、混合与政策

过滤器可以是硬规则、统计分类器，也可以是模型给出的质量分数。每个阈值都带来取舍。严格的质量过滤可能去掉 spam，却也可能丢弃罕见方言或专业材料；宽松的过滤可以保留广度，却会引入更多噪声。Karpathy 以语言过滤为具体例子：面向英文的数据集会有意减少非英文材料 ([Deep Dive, around 00:04:54](#))。

过滤后，不同来源会按选定比例混合。这个 mixture 不必与原始数据量一致：代码、数学、书籍或低资源语言可以被提高权重；规模很大但噪声较多的网页来源可以被降低权重。提高有限来源的权重，也可能让其中的样例在训练期间出现更多次。

这些选择把政策嵌入了数据。它们会影响哪些语言、领域、风格、观点和行为模式更频繁地贡献梯度。因此，模型能力并不均匀：在一个领域表现强，不代表它在另一个领域也同样强。

去重、频次与记忆

重复序列会获得重复的训练权重。完全相同的副本、近似重复的网页、引用段落和转载内容，因而可能让某些文本的出现频率远高于其表面上的语义重要性。

去重可以在文档、段落或更短片段层面进行。它能提高数据效率，并降低逐字记忆以及训练数据与评估数据重叠的风险。但去重并不完美：转述和部分复制的文档很难识别，而且有些重复结构是合理的。

因此，记忆并不是非有即无，而是不均匀的。模型可能复现一个出现过很多次的独特序列，却无法回忆只偶尔出现的事实。频次也不是唯一因素——模型规模、上下文、优化过程和序列本身的性质都会产生影响；模型还可以在不存储精确副本的情况下泛化模式。去重改变的是记忆发生的概率，并不能消除记忆。

覆盖范围、新鲜度与能力

覆盖范围有多个维度：主题、语言、时间段、体裁、代码生态、文化背景和难度层级。模型有时能泛化到训练样例之外，但稀少或低质量的覆盖通常只提供较少的可学习证据。非公开信息甚至可能完全没有进入语料。

数据收集日期有边界，而且并不均匀。名义上的训练 cutoff 是一种有用的简写，却不应被想象成一个整齐划一的时间点：不同来源可能在不同时间收集，预训练与 post-training 数据集也可能覆盖不同日期范围。Checkpoint 一旦完成训练，其参数就不会随着现实世界的变化自动更新。

因此，数据新鲜度与推理能力并不是一回事。增加推理过程不能揭示可用信息中根本不存在的事件。Post-training 还会进一步塑造模型表达哪些知识以及如何回应，但它无法让覆盖范围变得均匀。

Scaling Laws

在广泛的实验范围内，语言模型的 loss 与参数量、数据集规模和训练 compute 呈现近似的 power-law 关系。在 log-log 图上，当其中一种资源增加时，loss 中可降低的部分常常近似沿直线下降 ([Scaling Laws for Neural Language Models](#))。这意味着收益可预测但会递减：把某种资源乘以固定倍数，通常会让额外 loss 也乘以一个大致固定的比例，而不会带来没有上限的改进。

Scaling 变量彼此耦合。如果见到的 token 太少或优化算力不足，更大的模型也可能训练不足。没有足够的模型容量和算力，再大的数据集也无法被训练过程充分利用。数据质量和 mixture 同样会影响给定规模下最终达到的 loss。

Compute-Optimal Training

对于 dense Transformer，一个实用的一阶近似是：训练 compute 随参数量与训练 token 数的乘积增长。因此，在固定 compute 预算下，必须平衡模型规模和 token 数。

Chinchilla 研究发现，许多早期大模型相对于其参数量训练不足。在该研究的实验条件下，用更多 token 训练较小模型，可以在相同 compute 预算下取得更低的 loss ([Training Compute-Optimal Large Language Models](#))。常被引用的“每个参数大约对应 20 个训练 token”，是与特定模型、数据和假设相关的实验结果，并不是普适常数。架构、数据质量、数据重复、优化方法和目标本身，都可能改变最佳配比。

因此，“compute-optimal”必须说明所优化的目标。Chinchilla 式问题研究的是：如何在固定训练 compute 预算下最小化预训练 loss。它本身并不最小化延迟、内存占用，或部署后消耗的总 compute。

训练成本与 Inference 成本

训练会花费大量 compute 来产生一个 checkpoint。此后，每当 checkpoint 处理输入并生成输出，inference 都会继续消耗 compute。一个简化的全生命周期核算式是：

$$C_{\text{total}} = C_{\text{train}} + Q * C_{\text{inference_per_request}}$$

其中 (Q) 是请求数量。实际各项成本取决于参数量、架构、输入与输出长度、硬件、数值精度和 batching。

这一区别可能改变参数量与训练 token 数的理想配比。如果 checkpoint 会被使用很多次，对较小模型进行更多训练，有时能以更高的一次性成本换取更低的重复 inference 成本。请求很少时，或者必须使用较大模型才能达到所需质量时，取舍可能不同。Training-optimal 与全生命周期 compute 最优是两个不同目标。

Scaling 论断能说明什么、不能说明什么

Scaling law 描述的是 held-out cross-entropy loss 等总体趋势。它不保证每项任务、benchmark、语言或能力都会平滑改进。阈值化的 metric 可能把底层表现的渐进变化，变成看似突然出现的“emergent”跳变 ([Are Emergent Abilities of Large Language Models a Mirage?](#))。

观察到的行为还取决于 data mixture、架构、tokenizer、优化过程和 post-training，而不只是参数量。更大的模型不会因此获得训练数据收集之后的信息，更低的平均 loss 也不代表所有行为都会改善。经验 scaling 关系在其实际测量的范围内最可靠；远距离外推需要谨慎。

污染

如果评估样例、近似变体、答案或详细解题讨论出现在训练数据里，就会产生 benchmark contamination。公开 benchmark 及其答案在网上被大量复制时，重叠可能通过预训练或 post-training 数据进入模型。

污染会让分数既反映对评估材料的回忆，也反映对未见样例的泛化。普通去重无法完全解决这个问题：删除训练数据内部的重叠文档，不等于将训练数据与 benchmark 及其变体进行比对。污染并不会自动让每项结果失效，但它会限制仅凭 benchmark 分数能够得出的泛化结论。

要点

- 训练数据是由收集、过滤、混合和政策选择共同塑造的分布。
- 去重会改变有效频次和记忆风险，但无法消除所有重叠或记忆。
- 数据覆盖会随领域、语言和时间而变化，checkpoint 参数也不会自动更新。
- Scaling law 描述有用的总体趋势，而 compute-optimal training 会针对明确目标平衡模型规模与训练 token 数。

- 训练成本、重复 inference 成本和 benchmark contamination 共同限定了 scaling 论断的适用边界。

第 6 章：Inference 与 Sampling

Inference（推理）是指使用训练好的模型计算输出，而不更新模型参数。给定 token 上下文，模型会为词表中的每个 token 产生一个分数，这个分数叫作 **logit**。Decoding（解码）规则把这些分数转化为一次 token 选择。被选中的 token 会追加到上下文中，然后重复这一过程。

本章沿着这条路径，从 logits 一直讲到完整序列的生成。

Inference 计算什么

对于上下文 ($x_{1:t}$)，decoder-only Transformer 会产生下一个 token 的 logits 向量 (z)，词表中的每个 token 对应一个值。Logits 是相对分数，不是概率。Softmax 会把它们转化为概率分布：

$$p_i = \exp(z_i) / \sum_j \exp(z_j)$$

Logit 越高，概率也越高，但每个概率还取决于向量中的其他 logits。普通 inference 只通过已有参数执行前向计算；它不会运行反向传播，也不会更新 checkpoint。

上下文可以改变模型输出，却不改变模型参数。当前输入前面提到的事实可以影响下一个 token 的分布，因为模型会以该输入为条件。这样的临时条件化不同于把事实学习进权重，也不同于把它存入持久记忆。

从 Logits 到一个 Token

****Greedy decoding（贪心解码）****选择概率最高的 token。给定完全相同的 logits、变换和打破平局的规则，greedy decoding 会做出相同的下一 token 选择。不过，它仍可能产生重复或只在局部有吸引力的续写，因为每一步都会直接确定一个 token，而不会考虑所有可能的完整序列。

****Sampling（采样）****从概率分布中抽取 token，而不是总取最大值。Temperature（温度）通常按下面的方式改变分布：

$$p_i(T) = \text{softmax}(z_i / T)$$

其中 ($T > 0$)：

- `temperature=1` 时，logits 不会被重新缩放，因此 softmax 给出模型未经修改的下一 token 分布。
- `0 < temperature < 1` 时，分布变得更尖锐，高 logit token 获得更多概率。
- `temperature > 1` 时，分布变得更平坦，低 logit token 获得更多概率。
- `temperature=0` 在公式中没有定义，因为它会导致除以零。API 通常把它解释为 greedy 或 argmax decoding，但具体约定取决于供应商。

Top-p sampling 也叫 nucleus sampling（核采样）。它先按概率给 token 排序，保留累计概率至少达到 (p) 的最小集合，然后重新归一化并从这个集合中采样。Top-p 为 1 时通常不会移除任何 token；它本身不会让 decoding 变成 greedy。Nucleus sampling 的提出，是为了避免从不可靠的长尾中采样，同时保留随上下文变化的多样性（[The Curious Case of Neural Text Degeneration](#)）。

Temperature 和 top-p 经常结合使用，不过两者的具体应用顺序和边界行为取决于实现。随机性较低可以适合预期形式较窄的任务，而更宽的采样可以暴露不同续写。任何一种设置都不能决定内容是否正确。

自回归循环

生成是自回归的：每个被选中的 token 都会成为下一个 token 的上下文的一部分。简化后的过程如下：

1. 把输入上下文编码成 tokens。
2. 运行模型，得到下一个位置的 logits。
3. 应用已配置的 logit 变换或约束。
4. 通过 greedy decoding 或 sampling 选择一个 token。
5. 把选中的 token 追加到上下文。
6. 重复以上步骤，直到触发停止条件。

对输入执行的初始计算通常叫作 **prefill**。随后一次生成一个 token 的阶段叫作 **decode**。实现通常会在 decode 期间复用 KV cache，而不是重新计算之前所有 token 的 attention keys 和 values；[第 9 章](#)会解释这一区别。

早期的 token 选择会改变上下文，因而改变此后的所有分布。这就是为什么两次采样生成可能在一个很小的初始差异之后显著分叉。

输出越长，需要的计算越多，因为每增加一个输出 token，就需要再执行一次 decode。原因不只是最终文本包含更多 token：只有先选择并追加 token (t)，才能生成 token (t+1)。

停止生成

生成可能因为以下几种原因停止：

- 模型发出 end-of-sequence token。
- 达到已配置的最大输出 token 数。
- 遇到已配置的 stop sequence。
- Serving API 因另一个有文档说明、且由供应商具体定义的原因结束生成。

许多 API 会返回一个 **finish reason**，用来区分正常停止、达到 token 上限或其他终止方式。字段名称和可能取值因供应商而异，因此必须按照相应 API 的约定来解释。

达到最大输出长度意味着序列可能被截断。即使一句话、代码块或结构化值尚未完成，最后几个字符看起来也可能很流畅。Stop sequence 同样是一个外部边界：根据 API 的不同，匹配到的序列可能不会出现在返回文本中。这两种情况本身都不能说明此前内容是否正确。

Logit 变换与 Constrained Decoding

Decoding 可以在选择 token 之前改变或限制下一 token 分布，例如使用 logit bias、重复或频率 penalty，以及通过 mask 让非法 token 的概率实际上变为零。

****Constrained decoding（约束解码）****在每一步应用这类限制，使不断增长的 token 序列始终是某种 grammar、JSON schema、正则语言或固定 label 集合下的有效前缀。由于文本单位与 token 并不总是对齐，decoder 必须判断哪些 token 续写能够继续满足约束，而不能只逐个字符检查。

只要实现正确，这些方法可以保证输出属于所支持的句法语言。它们不能保证 JSON 值在事实上真实、代码具有预期行为，或若干强制 label 中一定存在充分的答案。约束还可能掩盖不确定性：即使没有合适选项，模型也可能被迫做出选择。

约束生成讨论的是哪些 token 序列可以被发出。如何解释生成出的动作描述，或是否执行外部动作，属于另一个边界，由第 12 章讨论。

随机性与可重复性

Sampling 使用伪随机状态，因此即使概率分布相同，多次运行也可能选择不同 token。如果 API 暴露 seed，固定 seed 可能提高可重复性，但相关保证取决于供应商，也可能依赖模型版本、后端和其他 decoding 设置。

Greedy decoding 消除了采样随机性，但 temperature=0 并不能保证所有托管请求都得到逐字节相同的响应。名义上相同的请求仍可能产生略有差异的 logits，因为 batching、浮点 kernel、硬件或 serving 软件都可能发生变化。如果领先的两个 token 几乎并列，一个很小的数值差异就可能改变 argmax；最初的差异随后又会改变整个自回归路径。

关键区别是：

- 对于完全相同的 logits 和确定性的平局处理规则，argmax 会选择相同 token。
- 在 API 边界上看起来相同的请求，并不保证在每一种 serving stack 上都产生完全相同的 logits。
- 使用 sampling 时，可重复性还取决于伪随机状态和采样实现。

最后，可重复不等于正确。Greedy decoder 可以稳定复现同一个错误答案，而 sampled decoder 可能一次给出正确答案，另一次给出错误答案。

多次完整采样

Inference 过程可以不只生成一个 completion，而是根据同一个输入生成 (k) 个完整样本。每个样本都沿着自己的自回归路径前进，因此这个集合可能呈现不同的措辞、方法或候选解。

当分布中存在多个合理续写时，多次采样会有帮助，但它本身不会指出哪个候选最好。生成 (k) 个候选通常也会消耗大约 (k) 倍的生成 token，除非 batching 或提前停止改变了实际工作量。如何给候选打分、如何报告 pass@k，以及 evaluator 是否可信，属于评测问题，由第 13 章讨论。

要点

- Inference 把 token 上下文映射成下一 token 的 logits，而不更新模型参数。
- Greedy decoding 选择 argmax；sampling 从 temperature 和 top-p 等控制参数塑造的分布中抽样。

- `temperature=1` 不缩放 logits，而 `temperature=0` 是通常表示 greedy decoding 的 API 约定。
- 自回归生成每次追加一个被选中的 token，并在 end token、长度上限、stop sequence 或其他有文档说明的条件下停止。
- Constrained decoding 可以强制句法，但不能保证语义正确。
- 降低随机性可以提高可重复性，但确定性输出不一定正确，托管执行也仍可能变化。
- 多次完整采样以额外 inference 成本为代价，探索模型分布中的更多区域。

第 7 章：Post-Training

预训练教模型在广泛的文本分布上预测 token；post-training 则在更窄的数据或反馈上继续优化，使模型更常表现得像一个 assistant。Karpathy 的 introduction 把预训练和 fine-tuning 分开讲，并描述后续阶段如何让模型更适合对话和指令遵循场景 ([Intro to LLMs, around 00:14:29](#))。

这个区别有助于解释：源自相似架构的 base model 和 assistant model，为什么会表现得如此不同。Post-training 改变各种可能输出的概率；它可以有有帮助的回答、拒绝、特定风格和多轮交互约定更容易出现，但这些倾向并不是对正确性、安全性或完整性的形式化保证。

Supervised Fine-Tuning

Supervised fine-tuning，简称 SFT，会在输入与期望回答配对的样例上继续训练。样例可以包括指令与回答、对话、格式模式或领域任务。模型仍然通过 token prediction 学习，通常把 loss 集中在目标回答的 token 上，但新数据会把输出分布推向被演示的行为。

SFT 数据可以由人编写、由其他模型生成，也可以由人工数据和合成数据混合而成。数据在训练前还可能经过筛选、修改或标注。不同项目的数据构成并不相同，因此，纯人工数据或纯合成数据都不是 SFT 定义的一部分。

一个最简对话样例如下：

User: Explain gradient descent.

Assistant: ...

在大量此类样例上训练后，相似 prompt 更容易得到 assistant-like continuation。一个有用的心智模型是：模型会复现入选答案中的模式，包括语气、细节程度、请求澄清的方式，以及拒绝哪些类型的请求。它并不会因此获得答案编写者或选择者的意图和理解。

SFT 可以教会模型有用的任务行为，但成功模仿训练分布本身，既不能证明回答为真，也不能保证这种行为能泛化到每个新输入。

学得的交互约定

对话中的 role 和边界最终都必须表示为 token。Chat template 可以把 role 标签、消息分隔符、轮次结束符和其他 control token 序列化成一个序列。当这些模式出现在 post-training 数据中时，它们就成为模型所学分布的一部分。

因此，assistant model 可能学会一些约定，例如以 assistant role 回答、回应当前请求、在信息不足时提问澄清、使用熟悉的格式，以及在预期边界结束回答。如果训练数据对某些类别的请求包含拒绝样例，模型也可以学会 refusal pattern。

这些约定是统计规律，而不是在模型外实现的符号协议。其可靠性取决于训练分布和 inference 时给出的 context。熟悉的序列化方式可以更稳定地诱发学得的行為，而不同或含糊的序列化方式则未必如此。

偏好优化：RLHF、DPO 与 AI Feedback

SFT 会演示期望回答，但许多质量标准通过比较候选答案来表达，比编写唯一的理想答案更容易。在 InstructGPT 工作中，标注者编写 demonstrations、排序模型输出，并提供 preference data，用来训练 reward model，再通过 reinforcement learning from human feedback (RLHF) 优化 policy ([Training Language Models to Follow Instructions with Human Feedback](#))。

经典 RLHF 流程包含五个阶段：

1. 训练一个 SFT assistant model，或从这样的模型开始。
2. 针对 prompt 采样多个候选回答。
3. 收集人类 ranking 或 pairwise preference。
4. 训练 reward model 去预测这些偏好。
5. 优化 assistant policy 以获得更高的预测 reward，通常同时约束它不要离 reference model 太远。

Reward model 是另一个学得的模型。它把候选回答及其 context 映射成一个分数，近似 comparison data 中的偏好 ([Deep Dive, around 02:52:39](#))。Reference constraint 会限制 policy optimization 偏离起始行为的程度；如果没有这种约束，优化更容易把 policy 推进 reward model 不可靠的区域。

Direct Preference Optimization (DPO) 利用 preferred 和 dispreferred response，通过 classification-like objective 优化 policy，不再采用相同的独立 reward-model 训练和在线 reinforcement-learning loop ([Direct Preference Optimization](#))。它是另一种从偏好中学习的方法，而不是真理来源。

Preference judgment 也可以部分来自模型，而不完全直接来自人。例如，Constitutional AI 使用书面原则引导的反馈，从而减少部分 harmless training 所需的人工标注量 (Constitutional AI)。无论反馈来自人还是模型，其质量和覆盖范围都会塑造优化所强化的行为。

代理奖励与 Reward Hacking

学得的 reward 是设计者所关心质量的代理。人类偏好先被压缩成有限的比较数据，再由模型近似，因此得到的分数必然不完整。Karpathy 把 reward model 描述为 human preference 的有损模拟 (Deep Dive, around 03:00:54)。

当 policy 针对一个不完美的 proxy 被强力优化时，它可能找到分数很高、却不符合底层意图的输出。这就是 reward hacking (Deep Dive, around 03:04:11)。Policy 可能学会偏好答案的表面标记，变得不必要地冗长，过度赞同用户，或利用 reward model 的盲点。

Reference constraint 和其他 regularization 可以限制优化移动的幅度，但不能把 proxy 变成真理。Preference optimization 可以提高平均的主观评分，同时仍在训练比较未覆盖的区域引入系统性偏差。

可验证奖励与 RLVR

有些训练任务可以直接计算 reward。精确答案可以与已知结果比较，证明可以被检查，程序可以针对测试运行，游戏也可以返回分数。另一些质量——例如解释是否有帮助、语气是否合适——则更依赖主观判断。

Reinforcement learning with verifiable rewards 通常简称 RLVR，它使用这种可检查的结果作为 reinforcement signal。Karpathy 区分了可验证领域中的 reinforcement learning 与基于不可验证偏好的 reinforcement learning (Deep Dive, around 02:51:33)。对于适用的任务，可验证信号比 preference score 更容易大规模使用，也更少依赖主观判断。

验证仍然只覆盖 checker 实际测量的范围。通过 checker 并不能证明回答具备所有理想性质；在可验证任务上的提升，也不一定会均匀迁移到缺少同类反馈的任务。RLVR 可以强化得到可检查结果的问题求解行为，但不能证明生成的推理忠实或普遍可靠。第 8 章会讨论 inference 时的推理行为。

行为不等于能力

Post-training 会改变可观察行为，也可能改善、压制或重定向任务表现。因此，区分 behavior 和 capability 很有用，但 capability 并不是一个可以直接观察、固定不变的量。模型产生什么，取决于输入、学得的回答倾向、decoding 和具体任务。

一次拒绝表明，模型在该 context 中产生了学得的 refusal behavior；它本身并不能证明模型在其他所有 context 下都没有能力生成所请求的内容。反过来，一次成功回答不能证明能力足够稳健，自信的回答也不能证明底层主张为真。

同样的限制也适用于 safety behavior。Post-training 可以降低不理想输出的概率，但学得的 refusal policy 并不是硬性保证。第 10 章会讨论不确定性、幻觉和 refusal behavior，第 12 章会区分模型生成的动作与动作执行。模型外的正式控制属于配套教材 Agent Harness。

Full Fine-Tuning 与 Parameter-Efficient Fine-Tuning

训练目标和选择哪些参数参与训练，是两个相互独立的决定。Full fine-tuning 会更新 base model 的全部或几乎全部参数，并生成一个反映这些更新的新 checkpoint。由于知识和行为分布在许多参数中，针对一个任务的更新也可能改变模型在其他任务上的表现。

Parameter-efficient fine-tuning (PEFT) 会冻结大部分 base 参数，只训练一小部分参数。LoRA 是常见的 PEFT 方法，它学习选定权重矩阵的 low-rank update。其他方法会添加 adapter，或训练少量类似 prompt 的参数。这些方法减少了可训练参数的数量，也可以把 task-specific change 与 base checkpoint 分开存储，但其质量和 runtime 特性取决于具体方法与任务。

Full fine-tuning 和 PEFT 都通过训练数据与优化目标改变学得的行为。两者都不会把模型参数变成 live database，也不能提供持续更新且带有来源归属的事实。第 11 章介绍如何从外部集合中检索信息，第 14 章总结模型属性与外围系统之间的边界。

要点

- Post-training 通过额外优化，让 base model 更可能表现得像 assistant。
- SFT 数据可以由人工编写、合成或混合而成；它既教任务回答，也教序列化的交互约定。
- RLHF、DPO 和 AI-feedback 方法通过偏好引导行为，但其目标仍是 proxy，而不是真理。
- RLVR 为结果可验证的任务提供可检查 reward，但其信号只覆盖实际被验证的部分。
- 拒绝和成功回答都是对行为的观察，而不是关于能力、真实性或安全性的完整证明。

- Full fine-tuning 更新大部分或全部参数；LoRA 等 PEFT 方法只训练较小的参数集合。

第 8 章：Prompting 与 In-Context Learning

Prompting 是 inference-time conditioning：模型接收一个 token context，并基于该 context 生成 continuation。改变 context 中的指令、数据、示例或格式，就可能在不改变模型参数的情况下，改变各种 continuation 的概率分布。

In-context learning (ICL) 是模型根据当前 context 中呈现的模式调整行为的能力。这种适应只在当前 context 内有效；它不是权重更新，也不会变成持久技能或记忆。GPT-3 论文通过多个任务上的 zero-shot、one-shot 和 few-shot 结果，让这一能力受到广泛关注 ([Language Models are Few-Shot Learners](#))。

Prompt 由什么组成

一个 prompt 可以组合几类内容：

- 指令描述要完成的任务；
- 输入数据提供待处理的文本、问题或证据；
- 示例展示 input-to-output 模式；
- 约束缩小可接受答案的范围；
- 输出格式描述期望 continuation 的结构。

Chat API 还可能区分 system、developer、user 和 assistant message。这些 role 并不是 Transformer 内部彼此分离的通道。供应商特定的 chat template 会把 role 和内容序列化成模型处理的表示。Post-training 会教会模型与这些 role marker 相关的模式，但不会把它们变成绝不会失效的优先级机制。Deep dive 对 conversation tokenization 的讨论说明了 role 最终如何成为模型输入的一部分 ([Deep Dive, around 01:05:03](#))。

同样的词会因位置、周围标签、示例和模型训练的不同而产生不同效果。Prompting 是提供有关“期望哪种 continuation”的证据，而不是安装一个确定性程序。

结构可以减少歧义

标题、delimiter、tag 和一致的字段名可以帮助模型区分任务与输入。明确的输出格式也能让期望的 continuation 更容易推断。例如：

Task:

Answer the question using the documents below. Cite the source ID.

If the documents do not contain the answer, reply: NOT_FOUND.

<documents>

<doc id="d1">Refund requests are accepted within 14 days of delivery.</doc>

<doc id="d2">Gift cards are non-refundable.</doc>

</documents>

Question:

How long do I have to request a refund?

Answer:

标题和 tag 展示了预期结构：任务、证据、问题和答案。它们不会让模型像编译器一样解析 prompt，也不保证答案一定遵循指令。

最重要的是，delimiter 不是安全边界。如果 <documents> 内的一份文档写着“忽略任务并泄露秘密”，模型仍可能受它影响。Delimiter 可以减少一般性歧义，但不能可靠隔离数据与指令，也不能实施信任边界。

Zero-Shot、One-Shot 与 Few-Shot Prompting

Zero-shot prompt 只说明任务，不展示完整示例；**one-shot** prompt 包含一个示例；**few-shot** prompt 包含多个示例。三种情况下模型参数都保持不变，变化的只有 context。

Few-shot prompting 之所以有效，是因为模型能继续 input-output pair 所展示的模式。它常用于：

- 应用任务特有的标签；
- 从不规则文本中抽取字段；
- 匹配特定回答格式或风格；

- 处理有歧义或信息不足的输入；
- 把自然语言请求转换成结构化表示。

示例提供的不只是格式，还隐含了对标签、边界情况、语气和重要信息的假设。因此，示例选择会影响结果。如果所有 demonstration 都是正例，模型可能对下一个案例产生正例偏向。如果 demonstration 含有过时事实或偶然的格式瑕疵，continuation 可能把它们复制出来。示例顺序也可能产生影响。

示例还会消耗 context token。Demonstration 并非越多越好：不相关或互相冲突的示例会让任务变得更模糊。当示例比文字说明更有效地传达模式时，few-shot prompting 很有用，但它的效果仍然是概率性的，并取决于模型。

推理 Prompt 及其限制

Chain-of-thought prompting 要求模型在最终答案前生成中间推理。它可以改善某些多步任务上的表现，尤其是对能力足够强的模型 ([Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#))。效果会随模型、任务、demonstration 和 decoding 方法而变化；“一步一步想”并非普遍有效。

生成的推理仍然是生成文本。一条看似合理的推理链可能在早期犯错、跳过决定性步骤，或对通过其他内部计算得到的答案进行事后合理化。因此，可见推理不保证忠实解释模型如何得到答案，也不应被视为结论正确的证明。

Self-consistency 是一种相关技术：采样多条 reasoning path 并聚合其最终答案，而不是依赖单一路径 ([Self-Consistency Improves Chain of Thought Reasoning in Language Models](#))。答案一致可以改善部分 benchmark 的结果，但重复一致并不能确立事实。该方法还需要多次生成，因此消耗更多 inference compute。

推理模型与 Test-Time Compute

有些模型经过 post-training，学会在 inference 时为中间推理投入额外计算，它们通常称为 *reasoning model*。例如，DeepSeek-R1 把在可验证任务上进行 reinforcement learning 描述为获得强推理行为的一条路径 ([DeepSeek-R1](#))；[第 7 章](#)介绍了这类 post-training。

这是一种 *test-time compute*：困难 prompt 可能使模型在给出最终答案前产生更多中间 token。这些 token 可能可见、被隐藏，或由供应商提供摘要，但仍然会影响延迟和计算成本。额外要求这类模型显式输出 chain of thought，可能是多余的，也可能干扰它已经学到的行为。

更多推理可以改善模型对已有信息的使用，但它不是 grounding。它不会提供缺失的当前事实，不会让错误前提变成事实，也不会把生成的解释变成经过验证的证据。Reasoning model 仍然受其参数和当前输入的约束。

模型边界上的工具使用 Prompting

从模型视角看，工具使用首先仍是一个 prompting 与 generation 问题。工具定义、参数 schema，有时还有示例，会进入模型输入。模型随后可以生成如下结构化 call：

```
{
  "name": "get_weather",
  "arguments": {
    "city": "Boston"
  }
}
```

这段 JSON 只是模型输出。模型并没有联系天气服务，也没有改变外部状态。外部系统会解释这个拟议 call，决定是否以及如何执行，并可能在后续模型 context 中返回 observation。模型随后可以基于该 observation 生成下一段 continuation。

本章只关心工具描述和 observation 如何影响模型生成。执行循环以及“提出动作”和“执行动作”的区别见 [第 12 章](#)。工具接口和生产控制属于配套教材 [Agent Harness](#)。

Prompt Injection 是上下文混淆

Prompt injection 发生在不可信内容包含类似指令的文本，并以非预期方式改变模型行为时。检索页面可能要求忽略原问题，上传文档可能藏有命令，tool observation 或图片也可能携带与预期任务竞争的文本。

在模型层面，这些内容都会变成能够影响 next-token distribution 的输入。Role marker、“把这当成数据”等措辞和 delimiter 可能帮助模型推断预期层级，但不会在可信指令与不可信内容之间建立硬隔离。因此，只靠写一个更强的 prompt，或把不可信文本嵌套在 tag 里，无法解决 prompt injection。

模型可以通过训练抵抗许多此类指令，但这种抵抗是行为表现，不是 enforcement guarantee。任何涉及外部动作或受保护信息的安全保证，都必须在模型之外实施。第 12 章说明了这条执行边界；配套教材中关于 [system prompt](#) 和 [安全控制](#) 的章节介绍系统层处理方式。

Prompting 不能提供什么

Prompting 可以引出并引导训练期间学到的能力，但不会更新模型参数。它无法可靠创造模型原本不具备的能力，无法保证回答正确，也无法保证生成的推理忠实。

Prompt 也不是持久记忆。它只对一次模型调用构成条件。如果应用稍后再次发送先前的对话，那些文本会成为一个新 context 的一部分；模型自身并没有在调用之间保存状态。Context window 的限制及其后果见第 9 章。

最后，prompting 不会授予权限，也不会执行外部动作。一句“你可以访问这个账户”仍然只是输入文本，生成的动作描述也仍然只是输出文本。执行、持久状态和安全隔离是模型外围系统的属性，不是 prompt 所能提供的属性。

要点

- Prompting 改变 inference-time context，而不改变模型参数。
- In-context learning 使用当前 context 中的指令和示例塑造行为。
- 结构和 delimiter 可以减少歧义，但不是安全边界。
- Few-shot 和 reasoning prompt 可以改善某些任务，但效果是概率性的，并取决于模型。
- 更多 test-time reasoning 不会创造新证据，也不保证正确。
- Tool call 是模型生成的提议；外部系统负责执行。
- Prompt 不提供持久记忆、权限、执行或安全隔离。

第 9 章：Context Window 与 KV Cache

Context window 是模型在一次生成中能够条件化的 token 序列。它的上限通常约束 prompt 加上已经生成的 token，而不只约束 prompt。如果模型的 context 上限是 (C) 个 token，序列化后的 prompt 使用了 (n) 个，那么最多只剩 (C-n) 个位置可用于生成。Provider 还可能分别设置 input 和 output 上限，因此模型的架构上限与 API 实际接受的限制并不总是相同。

Context 是一次生成的临时输入。它不是持久状态，也不能保证模型会可靠使用其中的每个 token。

有限的 Token 序列

模型接收 token ID，其中包括 chat template 添加的 special token。它们都会占用序列位置。生成继续时，每个被选中的 token 都会追加到同一序列，并减少剩余容量。因此，tokenization 和不可见的格式开销既影响能放入多少输入，也影响还能生成多少输出；见第 2 章。

超出限制是容量问题：API 可能拒绝请求、截断一部分内容，或在配置的 output 上限处停止生成。低于限制只能说明序列放得下，并不能证明模型能够同样可靠地回忆、比较或推理序列的每一部分。

Karpathy 把 context window 描述为模型有限的工作上下文，也就是它生成时当前能看到的信息 (Intro to LLMs, around 00:32:42)。把它类比为 working memory 很有用，前提是不要误以为它具有持久性或可靠的随机访问能力。

Prefill 与 Decode

自回归 inference 包含两个计算特征不同的阶段。

Prefill

在 **prefill** 阶段，模型让 prompt 通过所有 Transformer layer。Causal mask 使 prompt 中的各个位置可以并行计算，同时每个位置仍然只能 attend 到自身和更早的位置。

对于长度为 (n) 的 prompt，标准 dense self-attention 的 attention 计算量按 ($O(n^2)$) 增长。优化后的 attention kernel 可以大幅减少中间内存读写和常数开销，但不会改变这种两两比较的基本 scaling。Prefill 产生用于选择第一个生成 token 的 logits，也会创建 decode 阶段将要复用的 key 和 value tensor。

因此，prompt 长度会直接影响 prefill 工作量，通常也会影响首 token 延迟。模型的其他部分、硬件、batching 和 serving 实现同样会产生影响，所以 token 数量不能单独决定延迟。

Decode

在 **decode** 阶段，模型每次选择一个新 token。在每个 layer 中，模型计算新 token 的 query、key 和 value。Query 会 attend 到已经为此前序列保存的 key 和 value，随后新的 key 和 value 被追加到 cache。

使用 KV cache 后，无需重复进行更早 token 的 projection 和 MLP 计算。不过对于标准 dense attention，新 query 仍然需要与不断增长的前缀比较。因此，序列长度为 (t) 时，单个 decode token 的 attention 计算量是 ($O(t)$)，并不是常数时间。在 (n) 个 prompt token 之后生成 (m) 个 token，这些 token 的 decode attention 总工作量约为 ($O(mn+m^2)$)。如果没有 cache，朴素实现会在每一步反复运行整个增长中的前缀，重复大量计算。

这个区别解释了两种常见延迟指标：prefill 在很大程度上决定首 token 延迟，而顺序执行的 decode 在很大程度上决定后续 token 的生成速率。

KV Cache 存储什么

在每个 attention layer 中，每个已经处理过的 token 都有一个 key vector 和一个 value vector。**KV cache** 保存这些 tensor，供后续 decode step 复用。它缓存的是某个精确 token 前缀的中间数值状态，而不是事实，也不是文本摘要。

对于每个活跃序列，它的大小近似随下式增长：

每个请求的 KV 元素数 =
 $2 * \text{layer 数} * \text{缓存 token 数} * \text{KV head 数} * \text{head dimension}$

还要乘以每个元素所占的 byte 数。Batch size、并行采样和 beam search 都可能成倍增加这项成本。Grouped-query attention 或 multi-query attention 等架构使用更少的 KV head，可以降低开销；sliding-window 或其他 attention 变体则可能改变必须保留哪些 token。

因此，更长的 context 会消耗更多 cache memory，也要求每个新 query 读取更多缓存状态。即使 model weight 能轻松装入内存，KV-cache 容量仍可能限制 batch size 和 throughput。

不能仅仅因为前缀描述了过期事实就称 cache “过期”。这些 tensor 仍然正确表示生成它们的精确 token；过期的是内容。如果前缀中的某个 token 改变，受影响的 hidden state 及其派生的 key 和 value 就必须重新计算。

Context、KV Cache 与 Provider Prompt Caching

这三个概念相互关联，但不能互换：

- **Context** 是允许模型条件化的 token 序列。
- **单次请求内的 KV cache** 保存本次生成中已经处理位置的 layer-level key 和 value tensor。
- **Provider prompt caching** 可以在不同请求之间复用共享前缀的 prefill 工作。

Provider prompt caching 是 API 和 serving contract，并不是模型有 context window 就必然具备的性质。它的适用条件、前缀匹配规则、保留时间、计费方式和延迟效果都取决于 provider；匹配通常要求开头的 token 完全相同。Provider 可能通过保留 KV 派生状态或其他内部方式实现这个功能。无论如何，它都不会扩大 context window、提高前缀的可信度，也不会把请求变成持久的语义记忆。

工作上下文不是持久状态

一次调用结束时，它的 context 不会自动成为后续调用可用的状态。Application 可以在外部保存信息，再把其中一部分放入之后的 prompt，但那是另一个系统机制。详细的 context management 和 memory 架构见 *Agent Harness* 第 2 章和第 3 章。

Long-Context 容量与可靠性

更大的标称窗口提高的是容量：能够放入更多 token。它不保证模型会均匀或可靠地利用这些 token。长输入会暴露多种经验性弱点：

- 对相关信息所在位置敏感；
- 相关材料被 distractor 包围时准确率降低；
- 难以整合分散在相距很远位置的证据；
- 在触及硬性 context 上限之前，任务表现就已经下降。

Context rot 常用来泛指 context 变长时出现的这类退化。它是一种经验行为，而不是单一机制，也没有对所有情况都成立的统一阈值。严重程度取决于模型、任务、序列长度、位置和周围内容。

Lost in the Middle 发现，在 key-value retrieval 和 multi-document question answering 任务上，当相关信息位于输入开头或结尾时，模型表现可能优于信息位于中间时 (Liu et al., 2023)。并非每个模型都会呈现完全相同的模式，但这个结果说明了核心区别：信息处于模型支持的窗口之内，不等于模型会可靠使用它。

因此，阅读 long-context 主张时必须区分概念。最大 token 数描述的是能够接受的序列长度；短 retrieval probe 上的表现描述的是一种行为。两者单独都不能证明模型能在该长度下对任意输入进行可靠的理解、回忆或推理。

Context 不是 Enforcement Boundary

模型会对收到的序列化 token 进行计算；它不会在序列不同部分之间提供可靠的信任隔离。Label、role marker 和 delimiter 可能影响行为，但不可信文本仍然可能影响生成，详见第 8 章。

因此，context 是输入面和攻击面，而不是 authorization 或 enforcement boundary。任何硬性的访问或行动边界都必须存在于模型之外。详细控制属于配套教材 *Agent Harness*。

要点

- Context-window 上限通常约束 prompt 加生成 token；provider 的 input 和 output 上限还可能带来额外约束。
- Prefill 处理 prompt，并构建每层的 key 和 value；decode 随后复用它们，顺序生成 token。
- KV cache 避免重复计算旧 token，但其内存随缓存序列长度线性增长，标准 decode attention 仍然要扫描不断增长的前缀。
- Context、单次请求内的 KV cache 和 provider prompt caching 是生命周期与 contract 都不同的概念。
- Long-context 容量不保证模型可靠使用每个已放入的 token。
- Context 是临时输入，不是持久记忆，也不是模型强制执行的信任边界。

第 10 章：知识、幻觉与不确定性

LLM 经常显得知识丰富，因为预训练把大型文本集中的统计规律编码进了模型参数。这些规律支持回忆、解释和泛化，却不构成权威事实记录。即使语气和格式很像可靠答案，模型仍可能生成错误、过时或缺乏依据的流畅文本。

Karpathy 在 introduction 中把幻觉（hallucination）作为核心失败模式提出：模型没有内置的真理 oracle，却可以生成看似合理的文本（Intro to LLMs, around 00:10:04）。幻觉并不只发生在完全没有证据时。模型也可能误读已提供的证据、引用其中错误的部分，或把主张延伸到证据支持的范围之外。

参数化知识不是数据库

参数化知识（parametric knowledge）是可以从学习到的权重模式中引出的信息。它不像数据库那样保证记录唯一、来源明确或数值最新。某个事实在训练数据中可能以多个相互矛盾的版本出现，也可能因为非常少见而只得到微弱表示，还可能在相关训练数据收集之后发生变化。

这也意味着，“存储”只是一种比喻。通常没有某一个参数或某一行与单个事实对应。相关的名称、日期、措辞和关联分布在整个网络中，并与当前 context 相互作用。同一事实换一种问法，可能在一次 prompt 中成功引出正确关联，在另一次 prompt 中却引出不同或错误的关联。

尽管如此，参数化知识仍然有用。重要的边界在于：来自参数的合理答案是一项预测，不是从权威来源读取的记录。

可信假答案如何产生

预训练目标奖励对可能 continuation 的预测，而不是逐项对照现实检查每个主张。训练文本也包含错误、迷思、虚构内容、过时材料，以及没有标注真假的自信陈述。Post-training 可以让答案更有帮助、更完整，但有帮助和完整都不保证事实性。

形如答案的 prompt 还会把可能的 continuation 进一步推向形如答案的回答。如果问题预设了一个不存在的事件、书籍或定理，顺着该前提继续生成，在统计上可能比质疑它更容易。如果在没有来源时要求精确引用，即使组合出来的文献并不存在，标题、作者列表、期刊和年份也都可能各自显得合理。

以下情况的风险尤其高：

- 所需事实非常少见或很新；
- 问题包含强烈但没有依据的前提；
- 没有提供来源，却要求精确引文或引用；
- 交互更奖励流畅完成或权威格式，而不是承认不确定性。

生成的文字可能很像百科条目、法律条款或研究文献。这样的权威外形会提高读者的信心，却不会提高答案的真实性。

模型层错误类型

错误或缺乏依据的答案可能来自多种机制：

- **回忆错误（recall error）**替换了错误的名称、日期、关系或细节；
- **知识过时（staleness）**复现曾经为真或曾经常见、但目前已经不再适用的说法；
- **混合拼接（conflation）**把不同人物、文档、事件或概念的属性混在一起；
- **模仿性错误（imitative falsehood）**复现人类文本中的误解和迷思。TruthfulQA 的设计目标正是检验模型能否给出真实答案，而不是模仿看似合理的人类错误（TruthfulQA）；
- **错误前提顺从（false-premise compliance）**接受没有依据的前提，并围绕它生成解释。例如，“X 为什么发生？”会把 continuation 引向原因，即使“X 是否发生”本身仍有待确认；
- **证据误用（evidence misuse）**与已提供材料矛盾、选择了错误段落，或推导出材料并不支持的结论。

这些类型会相互重叠，而且并非每个错误答案都能暴露其内部成因。它们仍然有用，因为它们说明，幻觉远不只是从空白 context 中编造事实。

Grounding 改变的是输入

Grounding 会把外部证据或 observation 放入模型的当前输入。这可以提高生成受支持 continuation 的概率，却不会重写模型参数，也不保证模型一定正确使用证据。模型可能忽略相关段落，偏向与之冲突的参数化关联，或从真实来源中做出没有依据的推断。

第 11 章介绍 retrieval-augmented generation，以及外部文本进入 prompt 的路径。第 12 章说明模型生成拟议动作与外部系统执行动作之间的独立边界。

不确定性与校准

不确定性不等于校准 (calibration)。从操作性定义看，如果一组预测被赋予信心水平 (q)，那么在具有代表性的这些预测中，约有 (q) 的比例正确，这组预测在信心水平 (q) 上就是校准的。例如，在被赋予 70% 信心的答案中，应当约有 70% 正确。因此，校准是要在大量案例上测量的性质，不是由一句自信表述就能确立的性质。

准确率与校准也不相同。模型可能平均准确率很高，却对许多错误答案都赋予 99% 信心；也可能准确率较低，但报告的概率更符合实际观察频率。校准会随领域、问题格式、prompt、decoding 设置和 post-training 而变化。

以下三个量不应混为一谈：

- **言语信心 (verbal confidence)** **是“可能”或“肯定”这样的措辞。它是生成文本，既可能反映表达风格，也可能反映不确定性；
- **Token 概率 (token probability)** **是模型为某个特定 next token 赋予的概率。它衡量表层 continuation 的可能性，并不直接等于完整主张为真的概率；
- **答案正确概率 (answer-correctness probability)** **是在已定义评判规则的任务上，对最终答案正确可能性的估计。标准语言模型回答不会自动提供这个量。

研究显示，模型有时可以估计自己是否知道某个答案，但这种能力无法完美迁移到所有任务和 prompt ([Language Models Mostly Know What They Know](#))。因此，自报或引出的信心只是关于模型行为的证据，不是对答案的独立验证。

认知性弃答是一种习得行为

认知性弃答 (epistemic abstention) **是指模型因为无法用足够的知识或证据支持答案而放弃作答。“我不知道”和“提供的信息不足”这样的措辞是输出行为，并非从某个专门的不确定性寄存器中直接读出的结果。

模型倾向于作答而不是弃答，并非只有一个来源：next-token 目标、预训练数据中形如答案的模式、prompt 措辞和 post-training 偏好都可能产生影响。Karpathy 的 deep dive 强调了这个问题的一种表现：回答 demonstration 通常包含自信、完整的回答，却很少出现“我不知道”的样例 ([Deep Dive, around 01:20:32](#))。这会强化形如答案的 completion，但只用 SFT 并不能完整解释幻觉。

Post-training 可以显式奖励模型在超出已展示知识边界的问题上弃答。这种学得的行为仍不完整：模型可能面对陌生的知识缺口却没有弃答，也可能在原本能够回答时弃答。因此，弃答是一种有用行为，却不能证明模型完美探测到了内部知识边界。

安全拒答与 Jailbreak

安全拒答 (safety refusal) **是指请求落入学得的不安全或不允许行为类别时，模型拒绝执行该请求。它不同于认知性弃答：模型可能完全理解一个请求，却出于安全原因拒答；一个无害问题也可能只因为答案未知而需要弃答。

安全拒答受 post-training 和 context 塑造，因此是概率性行为，不是硬性保证。**Jailbreak** 是试图诱导模型绕过这些学得的安全约束，例如通过角色扮演、假设语境或编码指令来重构请求。Karpathy 在 introduction 中用 jailbreak 例子说明 refusal behavior 可能受到 context 干扰 ([Intro to LLMs, around 00:46:16](#))。

Jailbreak 与 prompt injection 有关，但并非同义词。Jailbreak 由“绕过行为约束”这一目标定义，可以直接来自用户。[Prompt injection](#) 则发生在不可信内容提供了类似指令的文本，并让模型以违背预期任务的方式改变行为；它可能针对安全约束，也可能只是重定向一个原本无害的任务。当注入内容试图绕过安全约束时，两者才会重叠。

因为拒答是一种习得行为，有关访问或外部动作的保证必须存在于模型之外。[第 12 章](#)介绍执行边界，详细的系统控制属于配套教材 [Agent Harness](#)。

模型自我报告不是权威元数据

“你是什么模型？”、“你的训练 cutoff 是什么时候？”或“你能浏览网页吗？”这些问题，仍然是在要求模型生成另一个 continuation。答案可能反映 post-training 样例、当前 prompt，或 application 提供的信息；它不是对部署配置的内省式读取。

自我报告可能正确，尤其是在 context 中已有准确 metadata 时，但这段生成文字本身不能确立其准确性。模型身份、发布信息和部署能力是模型 artifact 与外围系统的属性，不应从模型的文字中推断。

引用形幻觉

引用尤其有说服力，因为它的格式暗示背后存在证据。即使没有对应来源，模型也可能生成看似合理的标题、作者、期刊、法条编号、URL 或引文。它还可能引用真实存在但并不支持相关主张的来源，或把有效引用放到错误的句子后面。

以下三个问题彼此独立：

1. ****来源存在性 (source existence)**：**被引用的来源是否存在？
2. ****主张支持性 (claim support)**：**该来源是否真的支持对应主张？
3. ****引用附着关系 (citation attachment)**：**引用的格式和位置能否明确指出它支持的主张？

一条格式精美的引用，单凭外观无法回答以上任何问题。[第 11 章](#)介绍检索和来源追踪，[第 13 章](#)介绍如何评估事实与引用的正确性。

模型失败与系统失败

引入外部证据或 observation 后，最终答案错误并不能自动证明内容是模型编造的。如果模型忠实总结了一条错误 observation，失败来自上游来源或系统，即使最终主张仍然是错的。如果 observation 正确，模型却与之矛盾或添加了没有依据的主张，那么这部分属于模型失败。两者也可能出现在同一个答案中。

这一概念区别很重要，因为 grounding 改变了模型收到的内容，并没有把模型和外围系统合并成同一个组件。诊断完整的证据、工具、权限和执行路径属于系统主题，由配套教材 [Agent Harness](#) 展开。

要点

- 参数化知识是分布式且会犯错的，不是权威数据库。
- 流畅的假答案来自预测式训练、不完美数据、形如答案的 prompt 和学得的答案偏好。
- 幻觉包括回忆错误、知识过时、混合拼接、模仿性错误、错误前提顺从，以及对已提供证据的误用。
- 校准需要跨案例测量；言语信心、token 概率和答案正确概率是不同的量。
- 认知性弃答与安全拒答服务于不同目的；两者都是习得行为，而非绝不会失效的内部信号。
- Jailbreak 针对学得的行为约束；prompt injection 描述不可信指令改变预期任务。
- 模型自我报告和引用形文本都不是权威 metadata，也不能证明主张得到支持。
- 错误的系统输出可能来自模型错误、错误的外部 observation，或两者兼有。

第 11 章：Embeddings 与 Retrieval

Retrieval 在 inference 时为模型补充外部信息。Karpathy 把 retrieval-augmented generation (RAG) 介绍为一种办法：与其只依赖模型参数，不如把相关文档带进 context ([Intro to LLMs, around 00:41:33](#))。RAG 的基础论文把参数化 sequence model 与通过 retrieval 访问的信息结合起来 ([Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#))。

RAG 改变的是模型输入，而不是模型权重。因此，它是否有用取决于两个不同的问题：retrieval 是否找到了所需 evidence，以及模型是否正确使用了这些 evidence？

为什么需要 Retrieval

训练期间学到的信息被压缩在模型参数中。这类参数化知识覆盖面广，在生成时可以直接使用，但它反映的是训练数据；源文档发生变化并不会自动更新这些知识。

Retrieval 采用另一条路径。在 inference 时，系统搜索一个外部集合，选择相关 passage，并把它们放入模型有限的 context：

- **参数化知识**：间接表示在模型权重中的信息。
- **检索信息**：保留在权重之外，并作为某次调用的输入提供给模型的信息。

外部集合可以在不训练新 checkpoint 的情况下更新，检索到的 passage 也可以保留 source label 以供引用。不过，retrieval 并不保证选中的文本相关或真实；把 passage 放进 context，也不保证模型会忠实使用它。

Retrieval Embeddings

Embedding 一词可以指彼此相关、但并不相同的表示：

- **Token embedding** 把一个 token ID 映射为进入 Transformer 的向量。后续各层会把这些初始向量变成与 context 和 position 有关的 hidden state。
- **Retrieval embedding** 把长度不定的 query 或 passage 映射为用于 similarity search 的定长向量。

两者不能互换。Retrieval model 的训练目标会让相关的 query-passage pair 倾向于获得相似向量。有些系统对两侧使用同一个 encoder，有些则使用不同的 query mode 和 document mode。因此，在 index time 与 query time 必须一致地使用该模型要求的 mode 和 preprocessing。

Dense retrieval 会比较 query vector 与已存储的 passage vector，常见分数是 cosine similarity 或 dot product。如果向量已经归一化，二者会产生相同的排序。高分表示它们按照该 embedding model 的表示彼此接近；这个分数并不是“passage 能回答 query”的校准概率。

让 query 与所有已存储向量逐一做精确比较，所需工作会随集合规模增长。HNSW、IVF 等 approximate nearest-neighbor (ANN) index 通过避免 exhaustive scan 来降低 search latency。由于采用近似搜索，它们可能漏掉 exact vector search 本会返回的向量。这种 index 层面的损失，不同于 retrieval model 把语义上无关的 passage 排在高位。

Dense、Lexical 与 Hybrid Retrieval

三种常见 retrieval 方法强调不同的信号：

- **Dense retrieval** 按 learned retrieval embedding 之间的相似度给 passage 排序。即使 query 与 passage 共享的词很少，它也可能匹配改写和概念关系。
- **Lexical 或 sparse retrieval** 根据 term-based representation 给 passage 排序。例如，BM25 会使用 term frequency、inverse document frequency 和 document-length normalization。它奖励词项匹配，但属于 relevance-ranking function，并不像 `grep` 那样执行 exact string matching。
- **Hybrid retrieval** 结合 dense 与 lexical 结果，常见做法包括对分数归一化后加权，或融合两边的 rank。

Lexical retrieval 通常擅长稀有名称、identifier、error code 和 exact phrase。Dense retrieval 通常擅长寻找用不同措辞表达的同一概念。Hybrid retrieval 可以同时保留两类信号，但 fusion rule 仍需评估：不同 retriever 的 raw score 并不会天然可比。

第二阶段 reranker 可以对较小的 candidate set 做更仔细的评分，例如使用同时读取 query 与 passage 的 cross-encoder。Reranking 可以改进已有 candidate 的顺序和 precision，但无法找回第一阶段 retrieval 从未纳入的相关 passage。

最小 RAG Pipeline

最小 RAG pipeline 包含 indexing phase 与 query phase。

在 index time:

1. 收集构成 retrieval corpus 的文档。
2. 把每份文档切成可检索的 chunks。
3. 保留 source、title、section 等 provenance。
4. 为每个 chunk 计算 retrieval embedding。
5. 把 chunk text、vector 和 provenance 存入 search index。

在 query time:

1. 使用 retrieval model 的 query encoder 或 query mode 编码 query。
2. 通过 dense、lexical 或 hybrid retrieval 搜索排名靠前的 candidate chunks。
3. 可以选择用计算成本更高的 relevance model 对 candidates 做 rerank。
4. 选择能放进模型 context 的 passages, 并将它们与 question 或 instruction 一起序列化。
5. 让模型基于这份扩充后的输入生成答案。

显式的 query embedding 步骤很重要: 即使 index 本身工作正常, 用不兼容的表示搜索 document vectors 也会降低 retrieval 质量。最终 prompt 可以保留 source label, 让答案能够指回相应 passage; 不过, 引用是否获得支持、事实是否忠实, 属于另外的评估问题, 详见第 10 章与第 13 章。

Chunking

整份文档往往太大, 而且主题太宽, 不适合作为 retrieval unit。Chunking 决定哪段文本共用一个 retrieval representation, 也决定哪些 evidence 可以被独立选中。

Chunk size 带来几项取舍:

- Chunk 太小, 可能会把 claim 与 explanation 或 caveat 分开。
- Chunk 太大, 可能混合多个主题、弱化 embedding signal, 并占用不必要的 context tokens。
- 忽略文档结构的边界, 可能会在不合适的位置切断 paragraph、API entry 或 code function。
- 缺少 source 和 section 信息, 会让检索出的文本更难解释或引用。

Structure-aware chunking 会尽量保留 section、paragraph、API entry 或 code symbol 等语义单元。Fixed-size window 实现简单, 在文档结构不可靠时也可能有用。

相邻 chunks 之间的 overlap 可以保留跨越边界的信息。更多 overlap 也会扩大 index, 并可能返回多个几乎相同的 passage, 从而占用 result slot 和 context。因此, overlap 是一项可调取舍, 并非越多越好。当重叠或重复 chunks 被一起检索出来时, deduplication 可以减少重复 evidence。

衡量 Retrieval

评估 retrieval 时, 首先需要一组 queries 和 relevance judgments, 用来标明每个 query 对应的有用 passages。不同指标回答不同的问题:

- **Recall@k** 是 top (k) 结果中出现的相关 passages 占全部相关 passages 的比例: $(|R_k \cap G| / |G|)$, 其中 (G) 是 relevant set。当每个 query 只有一个必需 passage 时, 通常也会把它报告为所有 queries 上的 top-(k) hit rate。
- **Precision@k** 是 top (k) 结果中的相关结果比例: $(|R_k \cap G| / k)$ 。
- **Mean Reciprocal Rank (MRR)** 对第一个相关结果所在 rank 的倒数取平均, 因此强调尽早找到一个相关结果。
- **Normalized Discounted Cumulative Gain (NDCG)** 奖励把高度相关的结果排在前面, 并且能处理分级而不只是二元的 relevance judgment。

这些 task metrics 不应与 **ANN recall** 混淆。ANN recall 比较 approximate index 返回的 neighbors 与 exact vector search 返回的 neighbors。一个 ANN index 可能拥有很高的 ANN recall, 但 exact search 与 approximate search 返回的 passages 对用户任务都无关。反过来, 如果被近似搜索漏掉的 exact neighbor 确实有用, 降低 ANN recall 也会降低 task Recall@k。

如果没有清晰定义 retrieval unit 与 relevance, 任何 metric 都没有明确含义。一个事实如果重复出现在多个 chunks 中, 那么 chunk-level recall 和 precision 反映的情况, 可能不同于系统是否找到了足以回答 query 的 evidence。

Precision、Recall、Latency 与 Context Pollution

Precision 和 recall 是正式的 relevance measure，并不只是“少取一些”和“多取一些”的同义词。增大 (k) 通常会提高或保持 recall，因为系统考虑了更多 candidates；随着较弱的 candidates 进入结果集，precision 则可能下降。具体变化取决于 query set 与 ranking。

Retrieval 还受 latency budget 约束。在 generation 开始前，query encoding、index search、hybrid fusion 与 reranking 都会增加耗时。ANN search 可以降低 index latency，代价是可能损失一部分 ANN recall。更大的 candidate set 会给 reranker 更多找到相关 evidence 的机会，但也会增加 reranking 工作量。这些取舍应该一起测量，而不是孤立优化某一项 metric。

选中的 passages 还会占用 context。无关、过期、重复或彼此冲突的 passages 可能干扰模型，或引入缺乏支持的替代说法。这就是 **context pollution**：retrieval 可能提高 Recall@k，却同时让最终输入更难利用。因此，即使包含的 relevant chunks 更少，更小、更干净的结果集仍可能表现更好。

检索到的文本也可能包含 instruction 或 adversarial content。模型层面的 prompt-injection 问题在[第 8 章](#)讨论；与 query 相似，并不代表某段 passage 应被当作 instruction。

RAG vs Fine-Tuning

Retrieval 和 fine-tuning 改变的是不同部分：

- 当任务依赖规模较大、独立于模型而变化的信息集合，或答案需要指向 source passage 时，使用 retrieval。
- 当目标是改变跨调用反复出现的 behavior、style 或 task convention 时，使用 fine-tuning。

Fine-tuning 不是让持续变化的文档集合保持最新的可靠方法，而 retrieval 本身也不会教会模型一种新的稳定行为。一个系统可以同时使用两者：fine-tuning 塑造模型如何回应，retrieval 则为当前 query 提供信息。

配套教材边界

本章只讨论面向模型的 retrieval 机制。生产环境中的 index maintenance，以及如何把 retrieval 集成到更广泛的 agent workflow，属于配套教材 [Agent Harness](#) 的范围。

要点

- Token embedding 是模型内部的 token representation；retrieval embedding 则表示用于 similarity search 的完整 query 或 passage。
- Dense retrieval 捕捉 learned semantic similarity，lexical retrieval 捕捉词项证据，hybrid retrieval 将两者结合。
- 最小 RAG 会同时嵌入 chunks 与 query、检索 candidates、可选地 rerank，并把选中的 passages 加入模型输入。
- Chunking 和 overlap 决定哪些内容可以被检索，以及会出现多少重复 context。
- Recall@k、Precision@k、MRR、NDCG、ANN recall、latency 与 context pollution 衡量 retrieval behavior 的不同部分。
- RAG 提供 query-time information；fine-tuning 改变反复出现的 model behavior。

第 12 章：推理、工具与 Agent

LLM 可以生成类似推理的文本，但一次模型调用只能使用它收到的输入，也不能独自在现实世界中产生效果。工具跨越了这条边界：模型输出所请求操作的表示，外部系统执行该操作，再把结果作为后续模型调用的新输入。

Karpathy 的 intro 把 tool use 和 retrieval 作为超越纯文本生成、扩展模型能力的方式来讨论 ([Intro to LLMs, around 00:41:33](#))。ReAct 这类研究系统展示了语言模型如何在 reasoning、action 和 observation 之间交替 ([ReAct](#))。

Toolformer 则探索模型如何学习何时调用 API、如何调用 API ([Toolformer](#))。

工具调用是模型输出

工具使用不会改变模型作为 token generator 的基本性质。在描述了可用操作及其参数的 context 下，模型可以生成普通文本，也可以生成结构化的调用表示。即使 constrained decoding 保证了某种结构，生成过程产生的仍是一个拟议调用，而不是操作执行本身。

所以“模型能浏览”只是简写。模型不会独立打开页面。外部 runtime 提供 browser-like operation，执行被选中的调用，再把得到的页面内容编码进后续 context。

最小 Call/Result 协议

这套逻辑协议需要 call identifier、operation name、arguments，以及用同一 identifier 配对的 result。例如：

```
assistant_call:
  id: c1
  name: get_weather
  arguments:
    city: Paris
```

外部系统执行操作后，可以返回：

```
tool_result:
  call_id: c1
  content:
    temp_c: 17
    sky: clear
```

这是对协议外部表示的 vendor-neutral 记法，并不是每种 API 的字面 wire format。Provider 可能把 call 和 result 序列化为 typed message、JSON object、special-token sequence 或其他 model-specific structure。模型在 post-training 中学习相应的调用模式；见第 7 章。

流式输出中尚未完成的表示还不是完整调用。表示完成后，syntax validity、schema conformance、semantic correctness 和 authorization 仍是不同问题。合法 JSON 可能包含错误字段；符合 schema 的参数仍可能写错城市；合理的请求也仍可能被禁止。只有外部系统能决定是否以及如何执行。

最小 Agent Loop

工具型 agent 可以归结为一个很短的循环：

1. 外部 runtime 把任务 context 和可用操作的描述交给模型。
2. 模型生成回复或结构化调用。
3. 如果输出是调用，runtime 决定是否执行。
4. Runtime 把结果作为 observation 放进新的 context。
5. 模型继续生成，直到产出 final response 或 runtime 停止循环。

Observation 不会更新模型参数。它之所以能改变后续生成，是因为它被加入了输入 context。因此，一连串模型调用加上外部执行可以表现得像 agent，尽管每次单独的模型调用仍然只是条件生成。

工具增加了什么

不同操作沿不同轴扩展模型：

- 外部知识操作可以提供检索到的或当前的信息；
- 精确计算操作可以完成计算或可重复的转换；

- **环境交互**操作可以读取文件、打开页面或检查其他 live state；
- **外部状态**操作可以在模型之外创建、更新或发送内容；
- **生成式操作**可以产出其他模态的 artifact。

原生多模态本身不是 vision tool。多模态模型可以把编码后的图像直接作为输入的一部分，而外部 vision 或 OCR tool 会执行一次独立操作，再返回文本或结构化 observation。无论哪种情况，模型接收的都是一种表示，而不是对现实世界的直接访问。

推理与行动

推理和行动扮演不同角色。类似推理的计算可以帮助模型利用当前 context 选择或排列步骤。行动可以获得该 context 中原本没有的信息，或请求改变外部状态。ReAct 强调的正是这种反馈模式：reasoning 指导 action，返回的 observation 再影响下一步。

这套协议不应被误认为忠实的 chain-of-thought 记录。有些系统暴露 reasoning-like text，有些把中间推理隐藏起来，还有些返回简短 summary 或 plan。这些是不同的 artifact；可见 plan 不一定是产生 action 的计算过程实录。能够可靠观察的协议边界是拟议 action 及其返回的 observation。记录这两者有助于区分不良调用与误导性或失败的结果，而无须声称能够访问模型的完整推理过程。

Reasoning model 可能会在回复或行动前使用更多 test-time compute。这可以改善模型对现有 context 的利用，却不会创造新证据、执行操作，也不会调用之间提供持久状态。

模型边界

从这个 loop 可以直接得到几个限制：

- 输出 call 不代表操作已经运行或成功；
- 一次模型调用不会自然地下一次调用保留任务状态；连续性来自再次提供的 context，或来自外部保存的状态；
- Tool result 会变成输入 context，它可能不完整、错误或带有对抗性。模型无法在数据与指令之间提供可靠的 trust boundary；见第 8 章；
- Authorization 和外部副作用属于执行调用的系统，而不是提出调用的模型。

这些事实构成了从模型基础到系统设计的桥梁。配套教材 *Agent Harness* 详细介绍[工具接口与 MCP](#)、[权限与 sandboxing](#)、[长任务状态](#)、[人工监督](#)，以及[computer use](#)与多模态 agent。

要点

- 工具调用是结构化模型输出，不是已执行的动作。
- 最小 agent loop 在模型生成、外部执行调用和返回 observation 之间交替。
- 推理可以塑造行动，而行动可以增加证据或改变外部状态。
- 执行、授权、持久状态和监督都存在于模型之外。

第 13 章：评估 LLM 行为

LLM 行为具有概率性，并且依赖上下文。Prompt、chat template、decoding 配置或采样到的 token 路径发生变化，都可能改变结果。即使名义上相同的请求，也未必产生相同输出。因此，评估需要跨任务、跨 trial 积累证据；单个出色答案或单次失败都只是很弱的证据。

Eval 分数总是以一组设置为条件：model checkpoint 或服务版本、输入、decoding 配置、任务分布和 grader。它描述的是模型在这些条件下表现出的行为，而不是模型某种脱离上下文的内在属性。

Benchmark 及其局限

公开 benchmark 测量的是特定的行为切片。MMLU 测试广泛的学术和专业知识，TruthfulQA 考察模型如何回答容易诱发常见错误观念的问题，HumanEval 和 MBPP 则通过可执行的 programming problems 测试代码生成 (MMLU, TruthfulQA, HumanEval, MBPP)。BIG-bench 和 HELM 展示了另一类更广泛的工作：跨许多任务和指标比较模型 (BIG-bench, HELM)。

这些分数是有用的比较信号，但适用范围有限。Benchmark 可能不同于真正关心的任务，可能过度强调某种格式，可能逐渐饱和，也可能受到训练数据污染。Prompting 和 decoding 选择同样会影响测得的结果。聚合分数提高，并不保证每个任务、语言或失败模式都得到改善。

需要使用工具或交互环境的 benchmark，测量的是模型与外围协议、系统的组合。它们可能很有价值，但分数不是对模型本身的单独测量。

代表性任务

Golden task set 是一组经过整理的输入和可检查的成功标准。它应该覆盖有代表性的普通案例、重要边界情况和已知历史失败。成功标准可以是精确答案、一组必要属性、可执行检查或清晰 rubric。Task set 应进行版本管理，以便把任务及期望结果的变化与模型行为变化区分开来。

玩具 prompt 对诊断有用，但不能证明模型在更广任务分布上的表现。Task set 应接近结果实际要被解释到的输入和输出要求。保留一部分 held-out tasks，也能降低反复迎合可见示例、却没有改善目标行为的风险。

Capability Eval 与 Regression Eval

Capability eval 问的是：在明确的 prompting 和 decoding 设置下，模型能够完成哪些任务。它会有意包含尚不能稳定解决的任务，从而同时暴露已经展现的行为和继续改进的空间。它提供的是这些条件下能力被引出的证据，而不是模型在所有可能 prompt 下能做什么的完整证明。

Regression eval 问的是：更换模型、prompt 或 decoding 设置后，过去有效的行为是否仍然有效。当某个 capability task 变得持续可靠后，它也可以成为 regression case。两类 eval 可以使用相似机制，但回答的问题不同。

重复 Trial

由于行为会在不同运行之间变化，应报告 trial 数量、成功次数或成功率，并在需要比较时报告不确定性区间。10 次中成功 8 次仍是一个不确定性很大的估计；两种配置之间的小幅差异可能只是噪声。

两个常用汇总指标回答不同问题：

- **pass@k** 是 (k) 次尝试中至少有一次成功的概率。它适用于可以生成多个候选、并从中选出一个成功候选的场景。
- **pass^k** 是 (k) 次尝试全部成功的概率。它适用于同一任务必须反复成功的场景。

如果各次 trial 相互独立，并且每次成功概率都是 (p)，那么 pass@k 为 $(1-(1-p)^k)$ ，pass^k 为 (p^k) 。现实中的 trial 可能相关，因此经验估计应说明任务和尝试是如何采样的。两个指标没有哪个天然“更诚实”；应该选择哪一个，取决于正在研究的行为契约。

应记录模型版本、完整输入、chat template、采样参数，以及 API 支持的 seed。固定 seed 可以减少一种变化来源，也有助于做成对的调试比较，但它不能保证 hosted inference 可复现。后端变化、数值非确定性和实现细节仍可能改变输出。如果目标是刻画行为波动，就应运行多次相互独立的 trial。

不要把每一次 pass/fail 翻转都称为 flaky eval。如果同一份已保存输出得到不同 verdict、validator 间歇性失败，或任务输入意外变化，那么不稳定的测量设施或 grader，应该修复它。如果在声明的设置下，模型的多次输出确实在正确性上不同，那么 eval 发现的是不稳定的模型行为；重复试验应测量并报告它，而不是让它消失。

Grading

不同输出需要不同 grader：

- **Deterministic 或 code-based grader** 使用精确匹配、结构化输出验证或可执行检查。它们速度快且可复现，但也能拒绝合理变体，或只能检查很窄的属性。
- **Human grader** 可以对模糊或主观输出应用领域判断，但速度更慢；rubric 不充分时，不同评审者也可能意见不一。
- **Model-based grader** 可以大规模应用自然语言 rubric，但其 verdict 本身也是概率性的模型输出。

这些方法可以组合使用。对 **deterministic check** 真正能够确定的属性使用它，并用清晰标注的示例校准人工或模型判断。例如，citation 格式合法不能证明来源存在，而来源存在也不能证明它支持对应论断。

Grader 是目标质量的 proxy，而不是 ground truth。Post-training 中的 reward model 也有相同的基本限制：它近似的是偏好，而不是直接观察偏好 ([Deep Dive, around 02:52:39](#))。Model grader 的模型、prompt、rubric 和分歧处理规则也都应该被记录。

作为 Proxy 的指标

当开发过程不断针对一个可见 metric 优化时，输出可能在 metric 上进步，却没有改善背后的真实目标。这就是第 7 章讨论的 reward hacking 在 eval 中的对应现象 ([Deep Dive, around 03:04:11](#))。

例如，summarizer 可能通过添加无关引用来提高 citation count；答案可能在保留事实错误的同时最大化 keyword overlap；model grader 也可能奖励精致冗长的表达，而不是正确性。Truthfulness、usefulness 或 clarity 之类的质量，都无法由单一 metric 完整规定。

应组合互补指标，使用平衡案例和 held-out cases，并定期人工检查 grader 意见不一致的样本。分数上升时，要检查新获奖励的行为是否真是该指标原本要代表的行为。

Model Eval 的终点

Direct model eval 可以测量给定 context 下输出中可见的属性，例如 task accuracy、factuality、citation support，以及 format 或 schema validity。应保留输入、输出、配置和 grader 结果，使单项判断可以接受检查。

当成功依赖工具执行、错误恢复、权限、副作用或外部环境的最终状态时，评估单元就不再只是模型，而是 system eval。完整 agent trace、环境状态判分、运行可靠性和 release gate 属于外围系统；参见 [Agent Harness 第 10 章](#)。

要点

- LLM 行为具有概率性并依赖上下文，因此 eval 结论需要代表性任务和重复 trial。
- 公开 benchmark 只能提供关于自身任务分布的有限证据，而不是普适模型排名。
- Capability eval 探索能够引出什么行为；regression eval 检查已经展现的行为是否仍然存在。
- pass@k 与 pass^k 回答不同问题，seed 不能保证 hosted inference 可复现。
- Deterministic、human 和 model grader 测量的都是 proxy，并各有不同失败模式。
- 对工具执行、权限、副作用或环境状态的评估属于 system evaluation，而不是 model-only evaluation。

第 14 章：操作性心智模型

全书归纳

LLM 是一个具有广泛习得能力的条件 token 生成器。给定编码后的 context，它会产生下一个 token 的概率分布；反复选择 token，最终形成文本或结构化提议。训练塑造模型参数，prompting 为某次推理提供条件，decoding 则决定如何选择续写。这些机制都不会让模型变成数据库、持久状态存储、外部事实的权威来源，或现实世界动作的执行者。

前面的章节逐步建立了这幅图景。第 1-7 章解释模型的核心机制，从 token 和 next-token prediction 一直讲到 attention、训练、推理与 post-training。第 8-13 章讨论 prompting、context、知识、检索、工具调用和评估，同时标出模型行为在哪里结束、系统责任从哪里开始。本章最后把这些边界归纳成一套操作性心智模型。

模型—系统责任边界

六类模型属性决定了边界应该画在哪里：

模型属性	模型能做什么	外部系统必须做什么
生成只以编码后的当前 context 为条件。	解释和转换当前 context 中提供的 representation。	选择、组装并序列化模型收到的 context。
Context 有限，而且模型自身没有持久状态。	使用当前调用中表示出来的信息。	在模型外持久化状态，并决定之后把什么重新引入 context。
参数化知识不完整，而且可能过时。	回忆习得的模式，并基于所提供的 evidence 推理。	从外部来源取得权威或当前事实。
能力分布参差不齐，而且生成具有概率性。	在广泛任务上产生有用的候选答案。	对要求精确或可靠的部分做验证；用评估测量行为，而不是假定评估本身会提高可靠性。
工具调用只是一种输出，不是外部效果。	提议动作或输出结构化参数。	授权、校验并执行动作，同时承担其后果。
模型可能混淆数据与指令。	遵循习得的指令和安全行为，降低——但无法消除——产生不安全提议的概率。	在模型外建立信任边界，并控制权限和现实世界效果。

这条边界并不是对模型是否“智能”的判断。它直接来自模型执行什么计算，以及它无法控制什么。

能力是参差不齐的

模型能力是参差不齐的，不是一个统一的旋钮。一个模型可能解出困难的多步证明，却数错单词里的字母；也可能写出合理的数据库 schema，却算错简单算术。这种“jagged”（参差）或 Swiss-cheese（瑞士奶酪）式能力分布意味着，在一个任务上成功，并不能证明它在邻近任务上也可靠。

Tokenization 可以解释一部分字符级弱点（见第 2 章），训练覆盖的不均匀可以解释一部分知识缺口（见第 5 章）。但更广泛的操作性结论是：能力判断需要具有代表性、可重复的证据，详见第 13 章；精确要求仍需要模型外部的验证。

从 Foundations 到 Agent Harness

本书解释这些外部责任为什么存在。配套教材《Agent Harness：实践者教材》解释系统可以如何实现它们：

- Context 选择与持久状态：《上下文是一种有限资源》和《压缩、记忆与子代理模式》。
- 动作与外部效果：《工具与 Agent-Computer Interface》。
- 信任、权限与隔离：《沙箱、护栏与安全自治》。
- 可靠性测量：《评估》。
- 模型选择、路由、成本与运维：《模型选择、路由与推理模型》和《AgentOps——成本、隐私与生产运维》。

同一条边界也适用于多模态系统：模型接收的是文本、图片、音频或视频的编码 representation，而不是世界本身；representation 选择见《Computer-Use 与多模态 Agent》，其中的信任问题见《沙箱、护栏与安全自治》。

最该记住的一句话

Karpathy 在 deep dive 结尾给出了一条务实提醒：把这些系统当作工具使用，但不要完全信任它们（Deep Dive, around 03:09:24, 03:30:42）。正确态度既不是否定，也不是崇拜，而是在利用模型广泛能力的同时，把权威性、持久性、验证和后果交给真正能够提供它们的系统。

如果只记住一句话，记住这一句：

模型预测 **token**；**harness** 负责 **context**、状态、工具、权限、验证和后果。

要点

- 把责任分配到模型—系统边界的正确一侧。
- 根据模型属性推导边界，而不是根据输出是否流畅来判断。
- 不要把一次有能力的模型回答误当成一个可靠的系统。

参考文献

本书主要基于下面两期源讲座，并为具体概念补充论文引用。

源讲座

- Andrej Karpathy, *Intro to Large Language Models*. YouTube.
https://www.youtube.com/watch?v=zjkBMFhNj_g
 - Andrej Karpathy, *Deep Dive into LLMs like ChatGPT*. YouTube.
<https://www.youtube.com/watch?v=7xTGNNLPyMI>
-

架构与训练

- Ashish Vaswani et al., *Attention Is All You Need*, 2017.
<https://arxiv.org/abs/1706.03762>
 - Tom B. Brown et al., *Language Models are Few-Shot Learners*, 2020.
<https://arxiv.org/abs/2005.14165>
 - Jared Kaplan et al., *Scaling Laws for Neural Language Models*, 2020.
<https://arxiv.org/abs/2001.08361>
 - Jordan Hoffmann et al., *Training Compute-Optimal Large Language Models*, 2022.
<https://arxiv.org/abs/2203.15556>
 - Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo, *Are Emergent Abilities of Large Language Models a Mirage?*, 2023. <https://arxiv.org/abs/2304.15004>
 - William Fedus, Barret Zoph, and Noam Shazeer, *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*, 2021. <https://arxiv.org/abs/2101.03961>
-

效率与量化

- Tim Dettmers et al., *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*, 2022.
<https://arxiv.org/abs/2208.07339>
 - Elias Frantar et al., *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*, 2022.
<https://arxiv.org/abs/2210.17323>
-

Tokenization

- Rico Sennrich, Barry Haddow, and Alexandra Birch, *Neural Machine Translation of Rare Words with Subword Units*, 2015.
<https://arxiv.org/abs/1508.07909>
-

Post-Training 与偏好

- Long Ouyang et al., *Training Language Models to Follow Instructions with Human Feedback*, 2022.
<https://arxiv.org/abs/2203.02155>
 - Yuntao Bai et al., *Constitutional AI: Harmlessness from AI Feedback*, 2022.
<https://arxiv.org/abs/2212.08073>
 - Rafael Rafailov et al., *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, 2023.
<https://arxiv.org/abs/2305.18290>
 - DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025.
<https://arxiv.org/abs/2501.12948>
-

Prompting、Retrieval 与 Tool Use

- Jason Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, 2022. <https://arxiv.org/abs/2201.11903>
 - Xuezhi Wang et al., *Self-Consistency Improves Chain of Thought Reasoning in Language Models*, 2022. <https://arxiv.org/abs/2203.11171>
 - Patrick Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. <https://arxiv.org/abs/2005.11401>
 - Nelson F. Liu et al., *Lost in the Middle: How Language Models Use Long Contexts*, 2023. <https://arxiv.org/abs/2307.03172>
 - Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, 2022. <https://arxiv.org/abs/2210.03629>
 - Timo Schick et al., *Toolformer: Language Models Can Teach Themselves to Use Tools*, 2023. <https://arxiv.org/abs/2302.04761>
-

Decoding 与 Evaluation

- Ari Holtzman et al., *The Curious Case of Neural Text Degeneration*, 2019. <https://arxiv.org/abs/1904.09751>
- Stephanie Lin, Jacob Hilton, and Owain Evans, *TruthfulQA: Measuring How Models Mimic Human Falsehoods*, 2021. <https://arxiv.org/abs/2109.07958>
- Saurav Kadavath et al., *Language Models (Mostly) Know What They Know*, 2022. <https://arxiv.org/abs/2207.05221>
- Dan Hendrycks et al., *Measuring Massive Multitask Language Understanding*, 2020. <https://arxiv.org/abs/2009.03300>
- Mark Chen et al., *Evaluating Large Language Models Trained on Code*, 2021. <https://arxiv.org/abs/2107.03374>
- Jacob Austin et al., *Program Synthesis with Large Language Models*, 2021. <https://arxiv.org/abs/2108.07732>
- Aarohi Srivastava et al., *Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models*, 2022. <https://arxiv.org/abs/2206.04615>
- Percy Liang et al., *Holistic Evaluation of Language Models*, 2022. <https://arxiv.org/abs/2211.09110>

术语表

Agent

智能体。把模型调用放入循环的系统：模型可以提出工具调用，外部系统执行该调用，观察结果再进入后续模型调用。执行、授权和持久状态仍位于模型之外。

Attention / Self-Attention

注意力 / 自注意力。让每个 token 位置按相关性加权读取上下文中其他 token 表示的机制。self-attention 是 Transformer 在序列内混合信息的方式。

Autoregressive Model

自回归模型。一次生成一个 token，并让每个新 token 依赖之前的 token。

Base Model

大规模预训练之后、面向 assistant 的 post-training 之前的模型。Base model 是强大的文本补全器，但不一定是有帮助的 assistant。

Byte Pair Encoding

一种 subword tokenization 方法，通过反复合并高频符号对来构建词表。它帮助模型用较小单元处理稀有词和未见词。

Chain-of-Thought

一种 prompting 模式，让模型在答案前生成中间推理文本。它能改善某些任务，但这些文本不保证忠实反映产生答案的实际计算过程。

Chat Template

把包含 system、user、assistant、tool 等角色的聊天对话，序列化模型实际接收的 token 序列的格式。

Context Rot

序列变长时，模型利用信息的有效程度可能出现经验性下降。其严重程度取决于模型、任务、长度、信息位置和周围的干扰内容。

Context Window

模型在一次调用中能够条件化的最大序列长度，通常由 prompt 与已经生成的 token 共享。它是临时工作序列，不是持久记忆。

DPO

Direct preference optimization。一种 post-training 方法，直接用偏好数据优化模型，无需单独训练的 reward model。

Embedding

文本、代码、图片或其他数据的向量表示。文本 embedding 常用于语义搜索和检索。

Epistemic Abstention

认识论弃答。模型因为未知或证据不足而不作答的行为。它是学习到的行为，不是模型内部知识边界的可靠读数。

Fine-Tuning

预训练之后的额外训练。LLM 场景中通常包括基于 demonstrations 的 supervised fine-tuning 或基于偏好的优化。

Grounding

基于外部证据约束生成，并把论断与证据关联起来。Grounding 改变模型输入，不会更新模型参数，也不保证模型会忠实使用证据。

Hallucination

看起来合理但不被现实或提供证据支持的生成文本。

Harness

模型周围的软件系统。它承担单靠 token 生成无法提供的责任，例如执行、持久状态、权限、验证和现实后果。

Hybrid Search

混合检索。把稠密向量检索与 BM25 等词法或稀疏检索结合起来。grep 一类字面或正则搜索是另一种精确文本信号，也可以与前两者组合。

In-Context Learning

模型在不改变参数的情况下，根据 prompt 中的指令和示例调整行为的能力。

KV Cache

Transformer inference 中缓存的 key/value 张量，用于避免重复计算之前的上下文。它能提升效率，但不是语义记忆。

Logits

模型在转换成概率之前，对可能下一个 token 给出的原始分数。

Next-Token Prediction

模型根据之前 token 预测下一个 token 的训练目标。

Parameters

模型学得的数值权重。它们编码训练中学到的压缩统计结构和行为。

pass@k

一种评测指标：用 k 次采样尝试中至少一次解出问题的占比。k 越大，越偏向那些多试几次就能做对的模型。

pass^k

一种评测指标：k 次采样尝试全部成功的问题占比。它衡量给定采样条件下的重复可靠性，而不是至少一次成功的机会。

Post-Training

预训练之后塑造模型行为的训练，例如 SFT、RLHF、DPO 或 Constitutional AI。

Pretraining

预训练。在大规模文本语料上做自监督 next-token prediction 的训练，产出 base model。后续每个阶段（fine-tuning、post-training）都在它之上构建。

Prompt Injection

不可信内容包含类似指令的文本，并以 harness 未预期的方式影响模型行为的失败模式。

Provider Prompt Caching

供应商在多个请求之间复用匹配 prompt 前缀计算的特定功能。它不同于单次请求内的 KV cache，匹配方式、有效期和计费规则均由供应商约定。

RAG

Retrieval-augmented generation。在 inference 时由外部检索步骤选择信息并加入模型输入；模型权重不会改变。

Reasoning Model

推理模型。经过 post-training，在困难任务上使用额外 test-time compute 的模型；训练中常使用基于可验证奖励的 reinforcement learning。更多推理 token 可以改善某些任务，但不会创造缺失事实或外部证据。

Retrieval Embedding

用于相似度搜索的固定长度 query 或 passage 向量。它不同于 Transformer 内部逐 token 使用的表示。

Reward Hacking

过度优化一个不完美的 reward 或 grading signal，使分数提高，但真实目标没有改善。

Reward Model

根据偏好数据或其他 proxy objective 给候选输出打分的模型。Reward model 有用，但可能只是人类判断的不完美模拟。

RLHF

Reinforcement learning from human feedback。用人类偏好数据引导模型行为的一种 post-training 方法。

Safety Refusal

安全拒答。模型因为请求属于被视为不安全或不允许的类别而拒绝响应的学习行为。它不同于认识论弃答，也不是执行层保证。

Sampling

从概率分布中选择输出 token。Temperature 和 top-p 等采样设置会影响创造性、稳定性和方差。

SFT

Supervised fine-tuning。用期望 input-output 行为样例训练模型。

Temperature

Decoding 参数，用来改变下一个 token 概率分布的尖锐程度。低 temperature 更确定，高 temperature 更多样。

Test-Time Compute

在 inference 时花更多 token、时间和金钱来改进困难答案，区别于训练期 scaling。推理模型是最常见的例子。

Token

模型处理文本的单位。Token 可以是单词、subword、标点、空白模式或 byte-level 片段。

Tool Call

模型输出的、由 harness 解释为请求执行外部操作的结构。

Top-p (Nucleus Sampling)

一种 decoding 设置：从累计概率刚好越过 p 的最小 token 集合中采样。它裁掉不太可能的长尾 token，同时让候选集随模型的置信度自适应。

Transformer

现代大多数 LLM 使用的基础神经网络架构，核心依赖 attention mechanism，而不是 recurrence。

素材映射

这份表把本书主要知识点映射到两期源讲座。时间点是近似值，因为原始素材来自 YouTube 自动字幕。

Intro to Large Language Models

时间点	主题	使用章节
00:00:24	“两个文件”：参数和运行参数的代码	第 1、14 章
00:01:35	参数作为磁盘上的权重；运行时代码执行模型	第 1、6 章
00:03:59	参数来自训练，而不是人工编程	第 3、5 章
00:05:16	大规模 GPU 训练和昂贵参数生产	第 5、6 章
00:10:04	幻觉例子：流畅文本可能是错的	第 10 章
00:11:40	Transformer 神经网络架构	第 4 章
00:14:29	预训练之后的 fine-tuning	第 7 章
00:18:01	两阶段框架：pretraining 和 fine-tuning	第 3、7 章
00:22:11	Reinforcement learning from human feedback	第 7 章
00:28:20	工具使用，包括浏览器类工具	第 12 章
00:32:42	Context window 是有限工作上下文	第 9 章
00:33:41	能看图、生成图的多模态系统	第 2、12、14 章
00:41:33	RAG 和工具增强	第 11、12 章
00:46:16	Jailbreak attacks 与 safety/refusal 行为	第 7、10 章

Deep Dive into LLMs like ChatGPT

时间点	主题	使用章节
00:00:28	聊天文本框背后是什么	第 1、14 章
00:01:28	数据集构建与 FineWeb 示例	第 5 章
00:03:42	Web 数据过滤和处理	第 5 章
00:04:54	语言分类与过滤	第 5 章
00:12:07	Tokens 和 tokenization	第 2 章
00:14:34	数据集变成很长的 token 序列	第 2、3 章
00:15:36	在 token window 上训练	第 3 章
00:23:24	Transformer 是 LLM 使用的神经网络	第 4 章
00:24:29	Transformer 中的 attention block	第 4 章
00:26:12	训练后的 inference	第 6 章
00:35:58	Loss 作为训练信号	第 3 章
00:40:11	GPU 成本和 practical constraints	第 5、6 章
01:05:03	对话/chat format 的 tokenization	第 2、8 章
01:20:32	幻觉、知识 vs 工作记忆，以及训练模型说"I don't know"	第 10 章
01:25:07	现代 assistant 的工具使用	第 12 章
01:33:39	通过工具弥补模型限制	第 11、12 章
02:51:20	RLHF 和 reinforcement learning 框架	第 7 章
02:52:39	Reward model 是独立神经网络	第 7、13 章
03:00:54	Reward model 是人类偏好的有损模拟	第 7、13 章
03:04:11	Reward hacking 与过度优化 proxy 的限制	第 7、13 章
03:09:24	把模型当工具使用，不要完全信任	第 10、14 章
03:09:57	音频和图片上的多模态模型	第 2、12、14 章